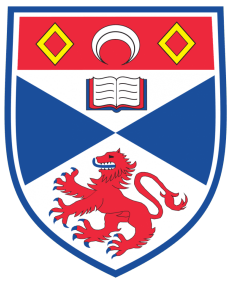
Verifying Language Model Reasoning Using Temporal Graph Constraints: A Structured Evaluation Approach

Hashim Iqbal



University of St Andrews
School of Computer Science

MSci Computer Science

Supervised by Dr Ognjen Arandjelović

May 2026

This dissertation is submitted in partial fulfilment of the requirements for the degree of Master of Science with Honours in Computer Science at the University of St Andrews.

Abstract

Large language models (LLMs) are increasingly used for tasks requiring multi-step temporal reasoning, yet existing benchmarks rarely test whether a model's own reasoning trace is internally consistent. Most evaluation protocols measure final-answer accuracy or relation-extraction F1, leaving the structural coherence of the generated reasoning path largely unexamined.

This dissertation presents a structured evaluation framework that extracts typed temporal graphs from free-form LLM outputs and verifies them against a constraint library and a graph-grounded fragment of Linear Temporal Logic (LTL) over step-indexed reasoning traces. The framework reports separate metrics for parse robustness, intrinsic graph validity, trace grounding, closure-level ordering fidelity, ambiguity discipline, and contradiction detection, with localised counterexamples identifying the step at which each violation first appears.

Five open-weight models in the seven-to-twelve billion parameter range are evaluated on a TempEval-3 platinum slice, a filtered MAVEN-ERE event-event slice and a controlled synthetic set for failure-mode probing. Four empirical findings emerge. First, the relationship between intrinsic invalidity and label-level error is model-dependent: it holds strongly for models whose dominant failure mode is structural, but requires qualification for models whose violations reflect reasoning-process failures dissociated from label correctness. Second, a model can achieve competitive label-level accuracy whilst being structurally invalid on a majority of tasks, a dissociation between output quality and reasoning-process integrity that label-only evaluation cannot detect. Third, several intrinsic axes are highly collinear on simple task structures, so multi-axis reporting must avoid overstating evaluation dimensionality. Fourth, a model can achieve perfect structural validity while failing contradiction detection entirely, demonstrating that intrinsic and task-level correctness must be assessed jointly. The framework, evaluation protocol, and failure taxonomy provide a reproducible artefact for structured analysis of LLM temporal reasoning.

Declaration

I hereby certify that this dissertation, which is approximately **14,700** words in length, has been composed by me, that it is the record of work carried out by me, except where credit is explicitly given to others by citation or acknowledgement, and that it has not been submitted in any previous application for a degree. This project was conducted by me at the University of St Andrews from **January 2026** to **May 2026** towards fulfilment of the requirements of the University of St Andrews for the degree of **MSci Computer Science** under the supervision of **Dr. Ognjen Arandjelović**.

In submitting this project report to the University of St Andrews, I give permission for it to be published online. I retain the copyright in this work.

Date: May 15, 2026
Signature:

Contents

Abstract 1
Declaration 1

1 Introduction **5**
1.1 Motivation 5
1.2 Problem Statement 6
1.3 Contributions and Objectives 6
1.4 Scope and Limitations 7
1.5 Dissertation Structure 8

2 Context Survey **9**
2.1 Temporal Logic and Formal Verification 9
2.2 Temporal Information Extraction and Temporal QA 10
2.3 Temporal Graph Learning 12
2.4 LLM Reasoning Limitations and Structured Reasoning Evaluation 13
2.5 Research Gap and Positioning 14

3 Requirements Specification **16**
3.1 Functional Requirements 16
3.2 Non-Functional Requirements 17
3.3 Out-of-Scope Requirements 18

4 Software Engineering Process **19**
4.1 Development Approach 19
4.2 Architectural Principles 19
4.3 Test-Driven Development 20
4.4 Tooling and Infrastructure 20

5 Ethics **21**

6 Design and Implementation **22**
6.1 Pipeline Overview 22
6.2 Structured Prediction Protocol 22
6.3 Temporal Graph Representation 24
6.4 Intrinsic Verification 24
6.4.1 Invariant Layer 25
6.4.2 Trace Construction 25
6.4.3 LTL Fragment 26
6.5 Gold Scoring 28
6.6 Failure Taxonomy 29
6.7 Benchmark Adapters and Rescore Operation 30
6.8 Analysis and Reporting Pipeline 30

7 Evaluation and Critical Appraisal **32**
7.1 Evaluation Against Original Objectives . . . 32
7.2 TempEval-3 Results . . . 33
7.3 Canonical Synthetic Results . . . 34
7.4 MAVEN-ERE Results . . . 35
7.5 Genuine LTL Signal . . . 36
7.6 Key Analytical Findings . . . 39
7.6.1 Necessary-but-Not-Sufficient Validity . . . 39
7.6.2 Axis Collinearity . . . 40
7.6.3 The Gemma Contradiction Paradox . . . 40
7.6.4 Mistral MAVEN-ERE Validity Collapse . . . 40
7.6.5 Cross-Dataset Rank Consistency . . . 41
7.6.6 RQ3: Verifier-to-Correctness Correlation . . . 41
7.7 Critical Comparison with Related Work . . . 42
7.7.1 LTLBench and TempoBench: Inverse Direction . . . 42
7.7.2 TG-LLM and Narrative-of-Thought: Graph from Text, Not from Output . . . 43
7.7.3 Graph of Verification: Convergent but Domain-General . . . 43
7.7.4 Comparison with TempEval-3 Published Systems . . . 43
7.7.5 Time-R1: Complementary Direction . . . 44
7.8 Threats to Validity . . . 44
7.8.1 Construct Validity . . . 44
7.8.2 Internal Validity . . . 44
7.8.3 External Validity . . . 45
7.8.4 Statistical Validity . . . 45
7.8.5 Dataset Validity . . . 45
7.9 Summary . . . 45

8 Conclusions **47**
8.1 Project Summary . . . 47
8.2 Drawbacks and Future Work . . . 47
8.3 Closing Remarks . . . 48

A Ethics Self-Assessment **54**

B Testing Summary **57**
B.1 Overview . . . 57
B.2 Test Modules and Coverage . . . 57
B.3 Test Strategy and Debugging Approach . . . 59

C User Manual **60**
C.1 Repository and Attribution . . . 60
C.2 Prerequisites . . . 60
C.3 Installation . . . 60
C.4 Lab Machine Setup . . . 61
C.5 Dataset Provenance and Preparation . . . 62
C.5.1 TempEval-3 . . . 62
C.5.2 MAVEN-ERE . . . 62
C.5.3 MATRES (future work) . . . 63

C.5.4 Canonical Synthetic Set 64
C.6 Quick Start: Single Model Run 65
C.7 Running a Multi-Model Sweep 65
C.8 Generating Analysis and Plots 66
C.9 Rescoring Existing Predictions 67
C.10 Validating a Dataset 67
C.11 Browser-Based Run Inspector 67
C.12 Dataset JSONL Format 68
C.13 Output Structure 68
C.14 Troubleshooting 69

D Intrinsic Axis Correlation Matrices **70**

Chapter 1

Introduction

1.1 Motivation

Large language models have demonstrated striking fluency across a wide range of natural language tasks, including multi-step reasoning, question answering, and narrative comprehension [1, 2]. Yet fluency is not correctness, and the persuasiveness of a generated reasoning trace carries no formal guarantee of its logical validity. This gap is particularly acute in *temporal reasoning* – tasks that require a model to commit to an ordered account of events, durations, or causal dependencies across time. A model may produce a locally coherent sequence of reasoning steps whilst simultaneously violating global ordering constraints: asserting, for instance, that event A precedes B in one step and that B precedes A in another, with neither contradiction surfacing in the final answer.

Standard evaluation methods are poorly equipped to detect such failures. Benchmarks for temporal reasoning typically measure final-answer accuracy or relation-extraction F1, neither of which exposes whether the reasoning path that produced a correct answer was itself internally consistent [3, 4]. The problem is not merely academic: as LLMs are deployed in settings where multi-step reasoning is load-bearing – legal document analysis, clinical timeline construction, intelligence summarisation – the absence of principled methods for verifying the structure of a model's reasoning constitutes a practical as well as a scientific shortcoming.

Chain-of-thought prompting [2] makes reasoning steps explicit, creating the possibility of structured inspection. This dissertation exploits that possibility: it treats a chain-of-thought output as a *step-indexed reasoning trace*, extracts a typed temporal graph from the free-form text, and verifies that graph against formal temporal constraints. The verification is post hoc and symbolic; it does not modify the model's parameters or require access to its internal representations. The result is an evaluation apparatus that complements, rather than competes with, approaches such as reasoning-tuning [5] or specialised temporal prompting [6].

1.2 Problem Statement

The central question this dissertation addresses is:

> Given a language model's free-form reasoning output on a temporal relation task, can a constraint-checking system determine whether that output is internally consistent as a temporal structure, localise any inconsistencies to specific reasoning steps, and report this in a way that is diagnostic rather than merely binary?

This question decomposes into four research questions that structure the dissertation:

RQ1 (Representation) How can natural-language reasoning traces be reliably converted into temporal graphs with explicit typed events and relations? (Sections 6.2–6.3)

RQ2 (Verification) Which temporal constraints – expressed as graph-structural invariants and as a graph-grounded LTL fragment – best capture the failure modes observed in LLM temporal reasoning outputs? (Section 6.4)

RQ3 (Predictive validity) Are verifier signals correlated with task-level correctness, and how does this predictive relationship vary across models and datasets? (Section 7.6.6)

RQ4 (Localisation) Can the verifier report the trace step at which an inconsistency first appears, and are the resulting counterexample objects useful for diagnosing reasoning failures? (Section 7.1)

1.3 Contributions and Objectives

The dissertation makes three contributions, ordered by their position in the framework's novelty claim.

1. **Graph extraction from free-form output** – the framework extracts a typed temporal graph from a model's own reasoning output, reversing the direction of travel relative to benchmarks such as LTLBench [7] and TempoBench [8], which supply graph-structured problems to the model and grade the final answer.
2. **Intermediate-trace verification** – verification is performed over the model's step-indexed reasoning trace rather than solely over the final answer. The LTL fragment is instantiated with formulas that check trace-level properties (final-commitment grounding, ordering-relation reversal across steps) invisible to answer-level evaluation.

3. **Categorical failure taxonomy** – the verifier produces a multi-axis report separating structural, grounding, trace, and LTL violations, rather than a single validity flag, revealing failure modes such as a model achieving perfect intrinsic validity whilst failing all contradiction items.

Each element has precedents in the literature; their combination does not, as detailed in Chapter 2. Delivering these contributions corresponds to three primary DOER objectives: (O1) design and implementation of a graph-grounded LTL fragment over step-indexed traces with decidable verification; (O2) a multi-axis evaluation protocol with coverage-aware closure F1 and a genuine-LTL decomposition separating trace-level from graph-level signal; (O3) empirical evaluation across five open-weight models on three evaluation sets (TempEval-3, a controlled synthetic set, and MAVEN-ERE [9]). Secondary objectives include the categorical failure taxonomy with counterexample localisation and a prototype visual interface supporting inspection of predicted graphs and their verification status.

1.4 Scope and Limitations

The framework is an *evaluation apparatus*, not a reasoning-improvement technique. It does not modify model behaviour, access model parameters, or provide inference-time guidance. All evaluated models are open-weight, locally served, and in the seven-to-twelve billion parameter range; results should not be generalised to proprietary or substantially larger models without independent replication.

The LTL component implements a graph-grounded fragment over bounded, finite traces. It supports the operators **G**, **F**, **X**, and **U** with Boolean connectives, but does not constitute general-purpose LTL model checking over unbounded state spaces. All claims involving formal verification refer to this fragment and are scoped accordingly throughout the dissertation.

The evaluation covers event–event temporal relation tasks using a four-label schema: BEFORE, AFTER, SIMULTANEOUS, and UNKNOWN. Absolute time inference, duration estimation, and interval-algebra relation sets are outside the scope of the current framework. This is a deliberate boundary, not a deficiency: the graph-based verification approach is well-matched to qualitative ordering tasks, where the relational structure of a temporal graph has direct semantic content. Extending the framework to handle numeric or calendrical temporal reasoning would require a substantially different scoring architecture and is identified as a direction for future work.

Three evaluation sets are used in the primary comparative analysis: TempEval-3 (496 tasks per model), the controlled canonical synthetic set (250 tasks per model), and a filtered MAVEN-ERE event–event slice (500 tasks per model).

The 496 tasks correspond to event–event TLINK pairs in the TempEval-3 Platinum test slice, with one task per gold TLINK after coarsening the six-relation TimeML inventory to the framework's four-label schema (BEFORE, AFTER, SIMULTANEOUS, UN-

KNOWN). The MAVEN-ERE experiment likewise uses a coarsened subset rather than the full source relation inventory: only event–event pairs with BEFORE and SIMULTANEOUS labels are retained, while relations such as OVERLAP, CONTAINS, BEGINS-ON, and ENDS-ON are outside the current four-label schema.

1.5 Dissertation Structure

Chapter 2 surveys the background literature across four themes and establishes how the three-contribution framing distinguishes this project from its closest methodological neighbours. Chapter 3 specifies the functional and non-functional requirements using MoSCoW classification, mapped to the DOER primary objectives. Chapter 4 describes the iterative, milestone-driven development process and architectural principles. Chapter 5 records the ethics self-assessment outcome. Chapter 6 describes the framework architecture and implementation: the structured prediction protocol, the temporal graph representation, the invariant constraint library, the LTL fragment, the gold-scoring methodology, and the implementation of novel algorithms including the balanced-brace JSON extractor, the union-find simultaneity closure, the memoised LTL evaluator, and the task-specific formula generator. Chapter 7 presents the empirical results across TempEval-3, the controlled synthetic set, and MAVEN-ERE, addresses all four research questions, critically compares the project to related published work, and documents threats to validity. Chapter 8 summarises the contributions, assesses the extent to which each research question and objective was achieved, and proposes directions for future work.

Chapter 2

Context Survey

2.1 Temporal Logic and Formal Verification

The formalisms used in this project have deep roots in two parallel traditions: temporal logic and qualitative constraint reasoning over time intervals. Pnueli [10] introduced Linear Temporal Logic (LTL) as a language for specifying properties of reactive systems, with operators for globally-holding (**G**), eventually-holding (**F**), next-step (**X**), and until (**U**) properties over a linear sequence of states. Allen [11] independently defined a qualitative interval algebra over thirteen base relations between time intervals, providing the vocabulary later adopted by temporal annotation schemes such as TimeML. Baier and Katoen [12] provide the canonical treatment of model checking: given a finite-state system model and a temporal property, automated algorithms determine whether the property holds and, when it does not, produce a counterexample trace pinpointing the violation. The counterexample idea is the direct precedent for the localisation mechanism in this framework: when a predicted temporal graph violates an LTL formula, the verifier returns the first reasoning step at which the violation is introduced.

The intersection of LTL and large language models has become an active research area since 2023. Cosler et al. [13] introduced nl2spec, a system for interactively translating natural language instructions into LTL formulae using LLMs. The direction of travel in that work – natural language to formal specification – is the inverse of the direction pursued here, where formally defined temporal specifications are checked against model-generated graphs. Pan et al. [14] proposed Logic-LM, which routes logical reasoning problems through a symbolic solver after an LLM translates the problem into a formal representation. Xu et al. [15] extended this to symbolic chain-of-thought, where each reasoning step is translated to a logical expression that can be executed deterministically. Neither work addresses the temporal domain specifically, and both still rely on LLMs to produce the symbolic form rather than verifying the consistency of a model's own temporal commitments.

Two recent benchmarks occupy the most directly adjacent methodological territory to this project. Tang, Nuamah, and Belle [7] introduced LTLBench, which samples temporal reasoning problems and corresponding LTL formulae, derives a ground-truth verdict using the NuSMV model checker, and then presents the verbalised problem to an LLM as a binary classification task. The direction is problem-to-LLM: a graph-structured

instance is constructed externally and the model is asked whether a given LTL formula holds over it. Holzer et al. [8] introduced TempoBench, which synthesises reactive systems from LTL specifications, presents the resulting automaton to a model, and asks it either to simulate execution traces or to identify causal inputs behind observed outputs. Again, the formal structure is supplied to the model as input.

Feng et al. [16] proposed VeriCoT, a neuro-symbolic framework that formalises each chain-of-thought step into first-order logic and uses SMT solvers to check consistency across steps. VeriCoT is the most methodologically similar recent verification work to this project, but it operates in a domain-general first-order logic setting rather than exploiting the specific structural properties of temporal graphs, and it does not report a categorical failure taxonomy separating hallucination, trace grounding, and ordering closure as distinct failure modes.

The distinguishing feature of the present work relative to all of the above is directional: temporal graphs are extracted from the model's own free-form reasoning output, not supplied to the model as input, and verification checks whether the model's own intermediate reasoning trace is self-consistent as a temporal structure.

2.2 Temporal Information Extraction and Temporal QA

Structured temporal reasoning in natural language processing has a long history of benchmarking, dating to the introduction of TimeML as an annotation scheme. Pustejovsky et al. [17] formalised the representation of events, temporal expressions, and temporal links (TLINKs) between them, providing a schema that is directly reflected in this project's typed graph structure: TimeML events become nodes and TLINK relations become directed typed edges. Strötgen and Gertz [18] demonstrated robust rule-based temporal expression normalisation with HeidelTime, establishing that canonical time anchoring is a tractable front-end component; the present pipeline inherits this motivation via its canonical relation mapping to {BEFORE, AFTER, SIMULTANEOUS, UNKNOWN}.

The TempEval shared task series produced the most widely used evaluation resources for temporal relation extraction. UzZaman et al. [3] defined TempEval-3, which remains the standard external benchmark for event–event temporal relation extraction on English newswire and is the primary external evaluation dataset used in this project. Crucially, UzZaman and Allen [19] introduced the temporal awareness F1 metric, which evaluates relation predictions against the closure of the gold annotation graph rather than against direct edge matches alone. The distinction between direct-edge correctness and closure-equivalent correctness, which is central to the multi-axis evaluation protocol of this dissertation, originates in that metric.

Ning et al. [20] introduced TORQUE, a reading comprehension dataset requiring multi-sentence ordering reasoning that significantly raised the difficulty of temporal QA beyond pairwise relation extraction. Tan, Ng, and Bing [4] introduced TempReason, which distinguishes three levels of temporal reasoning difficulty: time–time relations, time–event relations, and event–event relations, providing a structured decomposition that

informed the category design of the synthetic evaluation set used here.

The landscape of temporal QA benchmarking has expanded substantially since 2024. Chu et al. [21] presented TimeBench, a comprehensive evaluation covering symbolic, commonsense, and event-based temporal reasoning across a range of granularities, and showed that state-of-the-art models perform inconsistently across these categories. Wang and Zhao [22] introduced TRAM, which benchmarks a further range of temporal reasoning types and includes error analysis by reasoning category. Fatemi et al. [23] proposed Test of Time, which parameterises graph structure to isolate temporal reasoning from training-data memorisation by generating novel, unseen graph configurations; the methodological motivation for the author-constructed controlled synthetic dataset in this project is directly analogous. Su et al. [24] identified co-temporal reasoning about concurrent events as a distinct challenge not well captured by existing datasets, introducing the CoTempQA benchmark. Gruber et al. [25] recently released ComplexTempQA, a dataset of 100 million complex temporal QA items, demonstrating both the scale and the difficulty of the full temporal reasoning problem.

Wang et al. [9] released MAVEN-ERE, a large-scale dataset unifying event coreference, temporal, causal, and subevent relation annotations. Its temporal relation inventory (BEFORE, OVERLAP, CONTAINS, SIMULTANEOUS, BEGINS-ON, ENDS-ON) is richer than the TempEval-3 four-label scheme and provides a natural test of whether systems that perform well on pairwise ordering tasks generalise to a denser and more diverse annotation. MAVEN-ERE is the third external evaluation dataset used in this dissertation, alongside TempEval-3.

A recurring observation across this literature is that evaluation is conducted at the level of final answers: a model receives a passage and a question, and correctness is assessed against a gold label. No benchmark in this line of work asks whether the model's own intermediate reasoning about temporal relations is internally consistent. This is the specific gap that the present framework targets.

At the level of model capability, Liu et al. [5] recently introduced Time-R1, which fine-tunes a reasoning model on a curated temporal dataset using group relative policy optimisation and achieves strong results across multiple benchmarks. Time-R1 is an orthogonal contribution: it modifies model parameters to improve temporal reasoning capability, whereas the framework presented here assesses the temporal consistency of whatever output any model produces, without modifying the model. The two approaches can compose naturally, and Time-R1 provides a useful upper-bound reference for what capable temporal reasoning looks like.

Islakoglu and Kalo [26] introduced ChronoSense, which evaluates LLM temporal understanding using Allen-interval relations and reports that models handle these relations — including symmetric ones — inconsistently. ChronoSense is notable for demonstrating that models struggle specifically with transitive and symmetric inferences across Allen relations, providing an empirical motivation for the closure-level evaluation in this project. The present work's restriction to a four-label schema is a deliberate scope decision, consistent with the TempEval-3 protocol, and not an implicit claim that the interval algebra is unimportant.

2.3 Temporal Graph Learning

The term *temporal graph* carries different meanings in different research communities, and the distinction is essential for locating this project correctly within the literature. In the graph representation learning tradition surveyed in this section, temporal graphs are streams of time-stamped edges between entities, where each edge carries a numeric timestamp (in seconds, days, or similar units) and learning targets include future link prediction and dynamic node representation [27, 28]. In the natural language processing and knowledge representation tradition, temporal graphs instead encode *qualitative ordering constraints* between event intervals, following Allen's interval algebra [11]: edge labels such as BEFORE, AFTER, and SIMULTANEOUS specify relational constraints rather than positions on a numeric timeline. The TimeML annotation scheme [17] and the TempEval benchmark series [3] adopt the qualitative representation exclusively, as does this dissertation. This restriction is deliberate: qualitative ordering is the appropriate representation for narrative temporal reasoning tasks, where absolute timestamps are rarely available and the semantically relevant structure is the partial order over events. The graph representation learning literature is relevant here because it establishes the representational adequacy of directed typed graphs for capturing complex relational dynamics, and because it motivates the choice of a typed directed graph as the extraction target; the specific learning methods (memory modules, temporal attention) do not transfer directly to this project's symbolic verification setting.

The choice to represent temporal reasoning outputs as typed directed graphs connects this project to the literature on temporal graph learning, which studies how to build representations of time-stamped relational structures.

Xu et al. [27] introduced the Temporal Graph Attention Network (TGAT), which encodes temporal neighbourhoods using functional time encoding and multi-head attention. Rossi et al. [28] proposed Temporal Graph Networks (TGN), a general framework for learning from streams of temporal edges using memory modules and graph attention. Both works demonstrate that temporal edge structure carries information beyond what static graph representations can capture, a finding that motivates the explicit representation of reasoning steps as time-ordered states in the trace-indexed verification model used here. Gastinger et al. [29] extended the Temporal Graph Benchmark to knowledge graphs and heterogeneous graphs in TGB 2.0, providing the current standard for evaluating temporal graph learning methods. These graph learning methods are not themselves verification tools; their relevance is that they establish the representational adequacy of typed temporal graphs for capturing complex relational dynamics.

Within the NLP community, two lines of work couple language models directly with temporal graph representations. Xiong et al. [30] introduced TG-LLM (also referred to via the TGQA dataset), a pipeline that translates text into temporal graphs and then asks an LLM to reason over the graph. This is the closest published methodological predecessor to the present project. The critical difference is directional: TG-LLM constructs temporal graphs from text descriptions as inputs to the model, whereas this dissertation constructs temporal graphs from the model's own output and verifies their internal consistency. In TG-LLM, the graph is what the model reasons over; in this project, the graph is what the model produces, and verification determines whether it

is self-consistent. Zhang, Beauchamp, and Wang [6] studied temporal graph generation under a narrative prompting strategy and examined the faithfulness of the resulting graphs relative to the source text. That work shares the motivation of making temporal structure explicit and inspectable but does not apply constraint-based verification to the generated graphs.

The most closely adjacent work from 2025 is by Fang et al. [31], who proposed a framework that builds directed acyclic graphs from LLM reasoning steps and checks logical consistency across nodes. This work appeared after the development of the framework described in this dissertation and independently converges on the idea of graph-structured verification of reasoning traces. It differs from the present work in two respects: it operates in a domain-general logical reasoning setting rather than the temporal domain, and it does not report the multi-axis metric decomposition that distinguishes hallucination, trace grounding, and closure-level failures as separate evaluation dimensions.

2.4 LLM Reasoning Limitations and Structured Reasoning Evaluation

The large-scale language modelling paradigm established by Brown et al. [1] demonstrated that models could perform complex tasks in a few-shot setting, but the generation of fluent, plausible text does not guarantee logical correctness. Wei et al. [2] showed that eliciting intermediate reasoning steps through chain-of-thought prompting improves performance on a range of multi-step tasks. Chain-of-thought output creates the possibility of structured inspection: if intermediate steps are explicit, they can be checked. However, the conditions under which these steps faithfully reflect the model's underlying computation remain poorly understood.

Turpin et al. [32] demonstrated that LLMs frequently produce explanations that do not faithfully reflect the factors driving their predictions: biasing prompts can lead a model to produce a correct answer alongside an explanation that attributes it to the wrong reason. This is a fundamental challenge for any framework that treats chain-of-thought steps as genuine reasoning traces rather than as post-hoc rationalisations. The present framework addresses this in a specific and limited way: it checks internal structural consistency of the trace rather than claiming to verify that the trace reflects the model's actual computation. Lyu et al. [33] proposed Faithful Chain-of-Thought Reasoning, which translates each CoT step into an executable symbolic form and uses a deterministic solver to derive answers; the resulting traces are faithful by construction. This approach requires well-defined domain-specific translation rules and a suitable solver, constraints that are met in the temporal domain by the graph-constraint verification approach used here.

Dziri et al. [34] showed that Transformers fail compositional reasoning tasks by exploiting superficial pattern matches rather than performing systematic computation, and characterised these failures using computation graphs that expose where the model's reasoning diverges from a correct derivation. The graph-based failure characterisation in that work is closely analogous to the counterexample objects produced by the verifier

in this project. Mirzadeh et al. [35] demonstrated that mathematical reasoning in LLMs collapses under minor symbolic perturbations, reinforcing the argument that structured evaluation which controls for surface-form variation is necessary.

At the level of step-level verification, Lightman et al. [36] released PRM800K and showed that training a process reward model to grade each individual reasoning step outperforms outcome-only supervision on mathematical reasoning. This constitutes a strong existence proof for the value of step-level evaluation signals and motivates the trace-indexed structure of this project's LTL layer, which localises violations to specific reasoning steps rather than reporting only a final validity flag.

Lee and Hockenmaier [37] provide a recent survey of step-by-step reasoning trace evaluation, organising the literature around four quality axes: factuality, validity, coherence, and utility. The factuality and validity axes correspond closely to the gold-scoring and intrinsic verification layers in this dissertation's evaluation protocol, whilst the coherence axis motivates the trace-grounding constraint that checks whether each reasoning step's declared support edges are consistent with the model's final relational commitments. This taxonomy did not exist in published form at the time the framework was designed, and its convergence with the design choices made here provides post-hoc empirical validation of the decomposition.

The introduction of reasoning-tuned models such as DeepSeek-R1 [38] raises the question of whether extended chain-of-thought traces produced under reinforcement learning are more faithful than standard CoT. Chua and Evans [39] found that reasoning models are not consistently more faithful, with performance varying substantially by task type. This result is relevant because one of the five models evaluated in this project is DeepSeek R1 7B, a reasoning-tuned distillation, and the empirical results provide direct evidence for how a reasoning-tuned model's temporal trace consistency compares with standard instruction-tuned models on the same tasks.

2.5 Research Gap and Positioning

The four themes above converge on a shared gap. Temporal logic and model checking supply the formalisms and the counterexample paradigm, but their application to LLMs has so far graded final answers on externally constructed problems. Temporal IE and QA benchmarks have produced comprehensive datasets and established temporal awareness F1 as the closure-level metric, yet evaluation remains answer-level. Temporal graph learning has demonstrated the representational adequacy of typed temporal graphs, and recent work in TG-LLM shows LLMs can reason productively over externally supplied temporal graphs, but the self-consistency of the graphs a model produces in its own reasoning has not been assessed. The structured-reasoning evaluation literature has established both the faithfulness problem and the value of step-level signals, but without application to the temporal domain's specific constraint structure.

This project addresses the gap with the three-contribution combination stated in Chapter 1: graph extraction from free-form output, verification over step-indexed reasoning traces

with constraints exploiting temporal-graph structure (acyclicity, antisymmetry, closure consistency) and a graph-grounded LTL fragment, and a categorical multi-axis report distinguishing hallucination, unsupported references, closure-level failure, and contradiction from each other rather than a single scalar flag. The framework does not modify the model, require parameter access, or assume any particular decoding strategy; it is applicable to any model that can be prompted to produce structured temporal reasoning output, and its findings are diagnostic in the sense of Lightman et al. [36] and Lee and Hockenmaier [37]: they identify where and how reasoning fails, not merely whether the final answer is correct.

Chapter 3

Requirements Specification

This chapter captures the properties the framework must satisfy in order to address the four research questions and three primary objectives stated in Chapter 1. Requirements are derived from the project proposal, the plan-and-context survey, and supervisor feedback received during implementation. They are classified using the MoSCoW scheme: **M**ust have (essential for the core dissertation claims), **S**hould have (important for evaluation quality or reproducibility), **C**ould have (beneficial if schedule permits), and **W**ill not have (explicitly out of scope). Functional requirements address what the system does; non-functional requirements address how well it does it.

3.1 Functional Requirements

Table 3.1 lists the functional requirements. Requirements FR1–FR7 and FR10–FR11 are classified Must Have and correspond directly to the three primary objectives of the DOER: FR1–FR4 implement the bounded trace LTL verification layer (Objective 1), FR5–FR7 implement the novel verification metrics (Objective 2), and FR8–FR11 support large-scale empirical evaluation across models and evaluation sets (Objective 3). FR12 is a Should Have that enables verification logic to be updated and reapplied to existing predictions without re-running LLM inference, supporting the iterative refinement that characterises research software development.

Table 3.1: Functional requirements with MoSCoW priority and DOER objective mapping.

ID	Description	Prio.	Objective
FR1	The system shall prompt an LLM via a structured JSON template and receive a prediction containing an answer, a list of events, a list of typed temporal relations, and a list of step-indexed reasoning steps each carrying their supporting edges.	M	O1
FR2	The system shall parse the model's free-form output, extract the first well-formed JSON object even when the model wraps output in prose, and instantiate a typed temporal graph with directed edges labelled BEFORE, AFTER, SIMULTANEOUS, or UNKNOWN.	M	O1

ID	Description	Prio.	Objective
FR3	The system shall verify the extracted graph against an invariant constraint library covering acyclicity, antisymmetry, simultaneity consistency, duplicate edges, hallucinated nodes, reasoning grounding, and unsupported reasoning references.	M	O1
FR4	The system shall evaluate a graph-grounded LTL fragment over the step-indexed reasoning trace, supporting operators **G**, **F**, **X**, **U**, and Boolean connectives, with predicates *before*(a, b), *supports*(edge), *mentions_event*(e), and *has_violation*(kind).	M	O1
FR5	The system shall score predictions against a gold reference graph along two orthogonal views: direct-edge F1 (exact label match over canonical triples) and closure-equivalent F1 (after normalising AFTER, collapsing simultaneity groups, and computing the transitive ordering closure).	M	O2
FR6	The system shall report closure coverage (the fraction of non-UNKNOWN gold pairs for which the model made any ordering commitment) alongside all closure F1 figures, distinguishing a committed-only figure from a coverage-aware headline figure.	M	O2
FR7	The system shall produce localised counterexample objects for each violation, identifying the offending edge or step, the first violation step index in the reasoning trace, and the violation type.	M	O2
FR8	The system shall support evaluation of multiple models via a sweep manifest, running each model over a common dataset with uniform configuration and storing results in versioned output directories.	M	O3
FR9	The system shall validate dataset files against a defined JSONL schema before any inference run, reporting schema violations before consuming compute.	S	O3
FR10	The system shall record full provenance for each prediction: run identifier, model identifier, dataset version, code commit hash, and inference configuration.	M	O3
FR11	The system shall separate pipeline diagnostics (parse failures, transport timeouts) from reasoning quality metrics in all reports, so that infrastructure failures are not conflated with model behaviour.	M	O3
FR12	The system shall provide a rescore operation that re-runs verification on existing prediction files with updated verifier logic, without requiring LLM inference to be repeated.	S	O1, O2

3.2 Non-Functional Requirements

Table 3.2 lists the non-functional requirements. NFR1 and NFR4 are Must Have because reproducibility and a passing test suite are prerequisites for a defensible empirical dissertation. NFR2 is a Should Have rather than a Must Have because the primary constraint is correctness of the verification result rather than throughput; verification latency contributes negligibly to total runtime relative to LLM inference.

Table 3.2: Non-functional requirements with MoSCoW priority.

ID	Description	Prio.
NFR1	Experiments shall be driven by fixed configuration files specifying model, dataset path, random seed, temperature, timeout, and retry settings, and shall record enough provenance for reruns and comparison. Temperature zero and fixed seeds are used where supported, but local LLM serving is not assumed to guarantee byte-identical prediction files across repeated live inference runs.	M
NFR2	Verification (invariant checking and LTL evaluation) shall complete within 500 ms per task on the evaluation hardware, ensuring that the verification layer does not become a bottleneck in large-scale runs.	S
NFR3	The prediction parser shall implement a balanced-brace JSON extraction fallback to recover the first complete JSON object from outputs that include surrounding prose or markdown fences, recording repair attempts separately from genuine parse failures.	S
NFR4	The test suite shall pass in its entirety at each development milestone. New verification logic or scoring changes shall be accompanied by corresponding unit or integration tests before being merged into the main pipeline.	M
NFR5	Output directories shall be named with UTC timestamps and include the code commit hash in the run manifest, enabling any historical result to be traced to the exact codebase version that produced it.	M
NFR6	All aggregate analysis scripts shall operate on stored prediction files rather than requiring live model access, so that the full analysis pipeline can be re-executed offline after inference is complete.	S

3.3 Out-of-Scope Requirements

The following capabilities are explicitly excluded from the project scope. Their exclusion is a deliberate design decision rather than an implementation shortfall.

- **Absolute time and date inference** — The framework targets qualitative event-ordering tasks with a four-label relation schema. Tasks requiring numeric temporal reasoning (year estimation, duration calculation, calendar arithmetic) would require a substantially different scoring architecture and are out of scope.
- **General LTL model checking over unbounded state spaces** — The LTL component is a graph-grounded fragment operating over finite, bounded reasoning traces. It does not constitute a general-purpose model checker equivalent to NuSMV or SPIN. All claims of formal verification refer to this fragment.
- **Model parameter modification** — The framework is a post-hoc evaluation apparatus. It does not fine-tune, prompt-tune, or otherwise modify the models under evaluation.

Chapter 4

Software Engineering Process

4.1 Development Approach

The development followed an iterative, milestone-driven process suited to research software engineering, where requirements evolve with empirical findings and supervisor critique can redirect technical design between milestones. No formal agile methodology was adopted; as an individual project over a fourteen-week window the overhead of Scrum ceremonies would have been disproportionate. The work was instead organised around the external deliverables – the context survey, the interim report and the final submission in Week 15 – and two informal review points at supervisor meetings.

The two most consequential design changes, the introduction of the coverage-aware closure F1 and the addition of genuine LTL trace-level formulas, both arose from critique of intermediate results rather than from the initial specification. An iterative process accommodated both without destabilising the pipeline, because the modular architecture ensured that the scoring, verification, and analysis layers could be modified independently.

4.2 Architectural Principles

Three principles governed design decisions throughout development.

Separation of concerns: The pipeline enforces strict module boundaries between prediction parsing, temporal graph construction, constraint verification, LTL evaluation, gold scoring, and aggregate reporting. Each module exposes a typed interface. This separation meant that adding coverage-aware closure F1 required changes only in `src/run_summary.py`, and extending the LTL layer with task-specific formulas required no changes to the gold scoring path.

Reproducibility by construction: All experiments are driven by a JSON configuration manifest specifying model, dataset path, random seed, temperature, and timeout settings. Output directories are named with UTC timestamps and include the code commit hash. This supports reruns and comparison, but does not assume strict byte-identical output from repeated live LLM inference under local serving. The rescore operation allows verification logic to be updated and applied to stored predictions without repeating LLM inference, which was important when verification design evolved after data collection was complete.

Separation of pipeline failures from model behaviour: Parse failures and transport timeouts are recorded explicitly in the run summary and excluded from all reasoning quality metrics. This prevents infrastructure instability from silently distorting aggregate validity figures and is what makes the necessary-but-not-sufficient validity analysis in Chapter 7 possible.

4.3 Test-Driven Development

New constraints and modules were accompanied by tests before being considered complete. The suite reached 200 passing tests at submission, covering the LTL evaluator, all invariant constraints, closure and direct-edge scoring, the axis and correctness correlation analyses, and the dataset validation schema. This was particularly important for the verification logic, where failures are silent: an acyclicity check that misses a two-node cycle would distort validity rates across the entire evaluation without visible error. The suite also served as the primary regression safety net during iterative development; it was run in full after every module change.

4.4 Tooling and Infrastructure

The implementation language is Python 3.12. The temporal graph module uses a custom directed graph representation rather than NetworkX's standard, in order to maintain precise control over normalisation behaviour without depending on a general-purpose library's internal conventions; NetworkX is used only for transitive closure and cycle detection, where its correctness is well-established. LLM inference is served locally via Ollama [40], with all five models decoded at temperature zero and fixed seed 42 to try and make results as consistent as possible, though true determinism is impossible with stochastic models. Data analysis and visualisation use NumPy, SciPy, pandas, and Matplotlib. Version control was maintained throughout using Git, with the code commit hash recorded in every run manifest.

Chapter 5

Ethics

This project has raised no ethical concerns. The self-assessment form is included in Appendix A.

Chapter 6

Design and Implementation

This chapter describes the structure and implementation of the verification framework. Section 6.1 presents the four-stage pipeline. Sections 6.2–6.5 cover each stage, integrating algorithmic and data-structure details with the design decisions that motivate them. Section 6.6 documents the failure taxonomy; Section 6.7 describes the benchmark adapters, prediction-source abstraction, and rescore operation; and Section 6.8 covers the analysis and reporting layer.

6.1 Pipeline Overview

The framework operates as the four-stage pipeline shown in Figure 6.1: inference, parsing and normalisation, intrinsic verification, and gold scoring. At each stage a typed data structure is produced and stored; no stage mutates upstream outputs, so the pipeline can be resumed from any checkpoint and verification logic can be reapplied to stored predictions without re-running LLM inference – essential when verification design evolves after data collection. Analysis and reporting (`run_summary.py`) reads stored prediction files and produces aggregate metrics, plots, and bootstrap confidence intervals offline.

6.2 Structured Prediction Protocol

Extracting a typed temporal graph from a language model requires the model to emit structured output. The framework uses a single fixed prompt template, reproduced in Figure 6.2, that instructs the model to return a JSON object with four fields: `answer`, `events`, `relations`, and `reasoning_steps`. The template specifies exact key names, allowed relation labels, and formatting constraints; models are explicitly instructed to copy event names verbatim from the provided list, including the bracketed identifier (e.g., `"started [ei3]"`).

JSON extraction and repair: Models routinely wrap output in Markdown fences or add explanatory prose before and after the JSON block. The extractor (`_extract_first_json_object`)

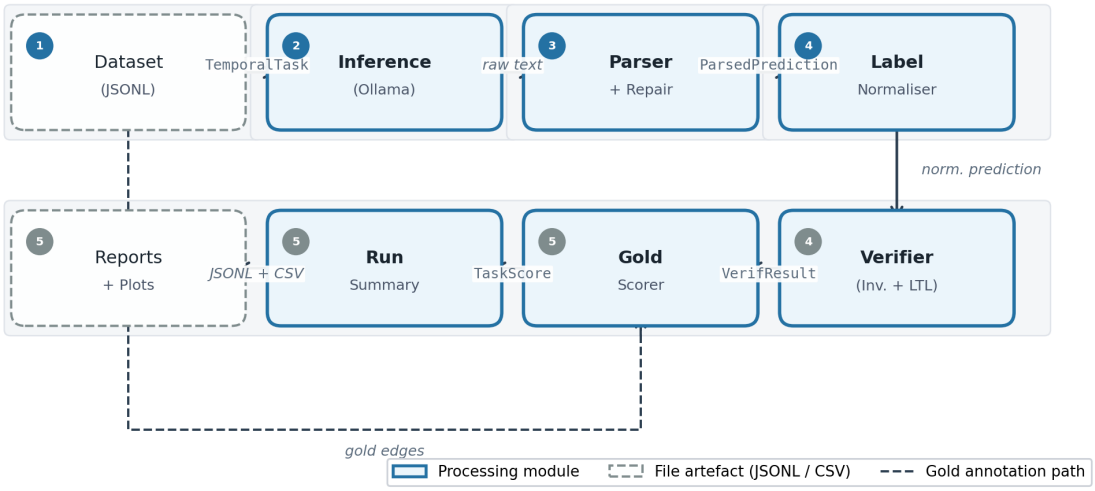


Figure 6.1: Four-stage pipeline. Rounded dashed boxes denote file artefacts; solid boxes denote processing modules. Every arrow represents a typed data handoff; no stage mutates upstream outputs.

```
{
  "answer": "string",
  "events": ["string", ...],
  "relations": [["Event A", "Event B", "BEFORE"], ...],
  "reasoning_steps": [
    {
      "step_id": 1,
      "text": "string",
      "supports": [["Event A", "Event B", "BEFORE"], ...]
    }
  ]
}
```

Figure 6.2: Structured prediction schema required from every model. The `relations` field defines the final temporal graph. Each `reasoning_steps` entry carries a `supports` list declaring which relations that step grounds. Allowed relation labels are BEFORE, AFTER, SIMULTANEOUS, and UNKNOWN.

uses a character-level state machine with a `depth` counter and an `in_string` flag: depth increments on `{` and decrements on `}`, but only outside string literals, with a one-character escape flag that suppresses interpretation of the next character. This correctly handles nested objects and strings containing curly braces, which a naïve regular expression cannot. If `json.loads` fails on the extracted text, a repair pass removes trailing commas before closing delimiters and auto-closes unclosed brackets via an explicit delimiter stack. Parse failures that survive both stages are recorded as `invalid_json` and excluded from reasoning quality metrics, so infrastructure failures do not distort aggregate validity figures.

Label normalisation: A systematic failure mode observed during development is that models abbreviate event identifiers: they emit `"started"` instead of `"started [ei3]"`. Without correction, every predicted node would be flagged as a hallucinated event and every edge would fail gold matching. The label normaliser (`evaluation.normalise_pred_labels`) resolves predicted labels against the task's canonical event list using three lookup strategies in order: exact string match, case-insensitive full match, and unambiguous trigger-word match (where the trigger word is the event text preceding the bracket). Labels matching no known event are passed through unchanged, so genuine hallucinations remain detectable by the verifier. Normalisation is applied to `pred_events`, both endpoints of every edge in `pred_edges`, and the support edges inside every `ReasoningStep`.

6.3 Temporal Graph Representation

The predicted relational output is represented as a `TemporalGraph`, a custom directed typed graph whose nodes are event strings and whose edges are triples (s, t, r) where $r \in \{\text{BEFORE}, \text{AFTER}, \text{SIMULTANEOUS}, \text{UNKNOWN}\}$. Internally, AFTER edges are normalised to BEFORE orientation during all structural computations, so that $\text{AFTER}(A, B)$ and $\text{BEFORE}(B, A)$ are treated as identical for cycle detection and closure computation.

Simultaneity groups and the collapsed ordering graph: SIMULTANEOUS edges define an equivalence relation over events. Computing the transitive ordering closure correctly requires collapsing these classes first, because ordering edges within a group interact non-trivially with inter-group edges: $A \sim B$ and $B \prec C$ imply both $A \prec C$ and $B \prec C$. A union-find structure (`_DisjointSet`) with path compression computes the equivalence classes, and the ordering graph is built over class representatives. NetworkX's `transitive_closure` is then applied to this collapsed graph and the resulting reachability pairs are expanded back to individual events. Path compression without union-by-rank is sufficient given the graph sizes encountered in practice (at most tens of events per task).

Key graph predicates: The verifier and evaluator use four graph predicates: `is_acyclic()` checks for directed cycles in the collapsed ordering graph; `direct_contradictions()` detects pairs (A, B) where both $\text{BEFORE}(A, B)$ and $\text{BEFORE}(B, A)$ appear as direct edges; `temporal_inconsistencies()` extends contradiction detection to the transitive closure; and `ordering_pairs()` returns the full transitive ordering closure used for closure-equivalent scoring.

6.4 Intrinsic Verification

Intrinsic verification asks whether a predicted temporal graph is self-consistent, grounded in the task's event set, and compatible with an active temporal specification. The verifier (`src/constraints.py`) operates in two layers: a structural invariant layer that checks

direct graph properties, and an LTL layer that evaluates a graph-grounded fragment over the step-indexed reasoning trace.

6.4.1 Invariant Layer

The invariant layer applies eight constraints partitioned across four sublayers, as shown in Table 6.1. The sublayer labels correspond to the `layer` attribute of each `InvariantSpec` and are used to decompose the aggregate validity signal: `graph_valid` is true when neither the intrinsic-temporal sublayer nor the formula layer fires; `trace_grounded` is true when the trace sublayer does not fire; both are reported as separate axes.

Table 6.1: Invariant constraints by sublayer, in evaluation order. Violations in the *grounding* and *trace* sublayers are recorded separately from *intrinsic-temporal* violations so that the two axes (`graph_valid`, `trace_grounded`) remain orthogonal. Grounding sublayer violations clear `trace_grounded` to false.

Sublayer	**Constraint**	**What it detects**
Format	`duplicate_edge`	Identical relation triples repeated in the output.
Grounding	`no_hallucinated_nodes`	Predicted events not present in the task event list.
	`reasoning_grounding`	Reasoning support edges referencing out-of-vocabulary events.
Intrinsic temporal	`antisymmetry`	Directly contradictory ordering pairs (A before B and B before A).
	`simultaneity_consistency`	Ordering edges between events within a SIMULTANEOUS group.
	`acyclicity`	Directed cycles in the collapsed ordering graph.
	`temporal_consistency`	Bidirectional reachability in the transitive ordering closure.
Trace	`reasoning_support`	Reasoning steps citing relations absent from the final prediction.

Each constraint implements an `evaluate(SpecContext) -> List[Violation]` interface. A `Violation` carries a type string, a human-readable message, a sublayer label, a `spec_source` field distinguishing invariant from LTL violations, and an optional `Counterexample` object containing the offending edges, implicated step identifiers, and notes. The typed `Counterexample` (rather than a free-form string) is a deliberate design choice: it allows the counterexample to be serialised, indexed, and cross-referenced with the reasoning trace without parsing.

6.4.2 Trace Construction

The LTL layer requires a step-indexed trace of the model's reasoning process. The trace is constructed by `build_temporal_trace` as a sequence of `TemporalState` objects, shown

in Figure 6.3.

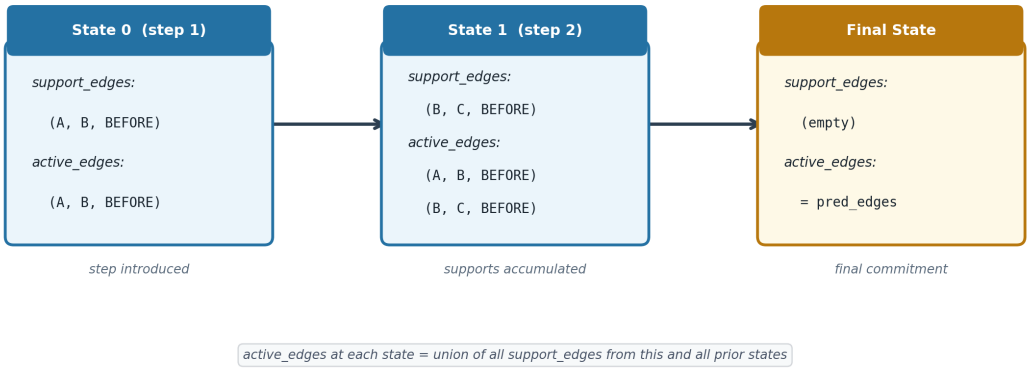


Figure 6.3: Trace construction for a two-step reasoning trace. `active_edges` is cumulative: each state's `active_edges` is the union of all `support_edges` from this and all prior states. The final state's `active_edges` is set to the model's `pred_edges`, not the accumulated supports, so the invariant layer checks the committed output rather than the intermediate reasoning.

Each state carries: `support_edges` (edges explicitly supported by that step's `supports` list); `active_edges` (the cumulative union of all support edges up to and including this step, representing the model's ordering commitments at this point in the trace); `mentioned_events` (events referenced in the step text or in `support_edges`); and `violation_types` (the set of invariant violation types recorded at or before this state, used by the LTL predicates). The final state is distinct: its `active_edges` is set to the model's committed `pred_edges` rather than the accumulated supports, its `support_edges` is empty, and `is_final_state` is `True`. The parser accepts `step_id` as either an integer or a string of digits, since some models emit `"1"` rather than `1`.

Violation propagation: The `has_violation` LTL predicate must hold cumulatively from the state where a violation first appears through all subsequent states. `TemporalTrace.with_violatio` implements this in a single forward pass, maintaining a running `seen` set and computing `introduced_violation_types` as the set difference at each state. This monotonicity ensures that a violation introduced at step i is detectable by the **G** operator at any step $j \geq i$.

6.4.3 LTL Fragment

The LTL layer evaluates a graph-grounded fragment of Linear Temporal Logic over the constructed trace, implemented in `src/ltl.py` as a recursive evaluator using memoised Boolean functions over trace indices. Supported operators are **G** (globally), **F** (eventually), **X** (next), **U** (until), and Boolean connectives $\neg$, $\wedge$, $\vee$. The fragment is decidable by construction over finite, bounded traces. Five predicates are defined: `before`(a, b)

holds at index i if $(a, b, \text{BEFORE}) \in \texttt{active\_edges}[i]$; $\texttt{supports}(a, b, r)$ holds if a semantically matching edge appears in $\texttt{support\_edges}[i]$; $\texttt{mentions\_event}(e)$ holds if $e \in \texttt{mentioned\_events}[i]$; $\texttt{has\_violation}(k)$ holds if violation type $k \in \texttt{violation\_types}[i]$; and `is_final_state` holds only at the final state index. Support matching uses the same canonical temporal-edge semantics as direct-edge scoring: $\text{AFTER}(a, b)$ is matched as $\text{BEFORE}(b, a)$, SIMULTANEOUS is order-insensitive, and UNKNOWN remains exact.

The default temporal specification instantiates five `FormulaSpec` objects in two categories (Table 6.2). Category I formulas re-express three invariant violation types as globally-holding trace properties; their contribution is temporal localisation, identifying the trace step at which a known invariant failure *first* appears. Category II formulas check properties invisible to the invariant layer, which operates only on the final `pred_edges`. Both Category II formulas are instantiated only when `reasoning_steps` is non-empty, since an absent trace provides no positive evidence to check.

Table 6.2: LTL formulas in the default temporal specification. Category I formulas corroborate the invariant layer over the trace; Category II formulas check trace-level properties that the invariant layer cannot detect.

Cat.	Name	Formula	Detects
I	`ltl_no_contradiction`	$\mathbf{G}(\neg\,\texttt{has\_violation}($ "contradiction"))	Contradiction persisting to final state
I	`ltl_no_temporal_inconsistency`	$\mathbf{G}(\neg\,\texttt{has\_violation}($ "temporal_inconsistency"))	Inconsistency persisting through trace
I	`ltl_no_hallucinated_nodes`	$\mathbf{G}(\neg\,\texttt{has\_violation}($ "hallucinated_node"))	Hallucinated event at any trace state
II	`ltl_unsupported_final`	$\mathbf{F}(\texttt{supports}(a, b, r))$ per committed edge	Committed edge lacking trace-level grounding
II	`ltl_trace_inversion`	$\mathbf{G}(\texttt{supports}(a, b, \text{BEF}) \rightarrow \mathbf{G}(\neg\,\texttt{supports}(b, a, \text{BEF})))$	Opposing ordering commitments across steps

Memoised evaluator: Without memoisation, evaluating $\mathbf{G}(\mathbf{F}(\phi))$ over a twenty-step trace has worst-case complexity $O(n^d)$ in trace length and formula depth, sufficient to cause perceptible per-task latency. The evaluator's inner `holds` function is a nested closure decorated with `@lru_cache(maxsize=None)`; because `Formula` instances are frozen dataclasses and therefore hashable, the cache keys on (*formula*, *trace index*) pairs for the lifetime of a single `evaluate` call. The failure-explanation traversal benefits from the warm cache: each sub-formula lookup returns immediately from cache, allowing the explanation to identify the minimal failing sub-formula and the earliest trace index at which it failed. This index is recorded as `first_violation_step` in the `VerificationResult` and propagated to the counterexample.

Task-specific formula generation: The two Category II formulas are generated dynamically per prediction by `Verifier._task_specific_formulas` and merged with the static specification inside `_formula_violations`. For each non-UNKNOWN edge

(a, b, r) in the final prediction, the method appends $\mathbf{F}(\texttt{supports}(a, b, r))$ after canonical temporal-edge matching, requiring that at least one reasoning step semantically supported the edge. For trace inversion, the method scans all step support edges to collect the set of canonical BEFORE pairs that appear anywhere in the trace, then for each pair (a, b) appends $\mathbf{G}(\neg\,\texttt{supports}(a, b, \text{BEFORE}) \vee \mathbf{G}(\neg\,\texttt{supports}(b, a, \text{BEFORE})))$, detecting mid-trace ordering reversals invisible to the invariant layer.

After rescoring all stored predictions, `ltl_unsupported_final_commitment` fires on 7.1% of DeepSeek R1 7B and 18.8% of Mistral 7B tasks on TempEval-3, rising to 57.4% for Mistral 7B on MAVEN-ERE – a cross-dataset progression (TempEval 18.8%, canonical 22.8%, MAVEN-ERE 57.4%) that demonstrates how this failure mode scales with task complexity. `ltl_trace_inversion` fires exclusively on datasets where multi-event reasoning chains are present: 16.4% (Qwen 3.5 9B), 19.6% (Llama 3.1 8B), and 13.6% (Mistral 7B) on the canonical set. The zero rate on TempEval-3 is structurally expected: pairwise two-event tasks with one or two reasoning steps offer no opportunity for a step to reverse a prior commitment.

6.5 Gold Scoring

Gold scoring is entirely independent of intrinsic verification. It compares the model's predicted edge set against the reference annotation along two orthogonal views, as illustrated in Figure 6.4.

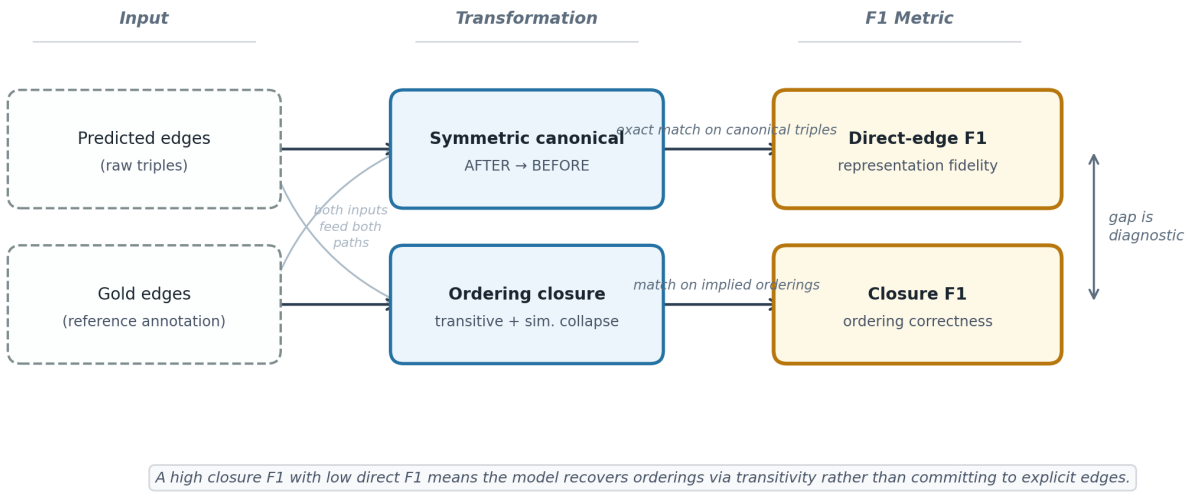


Figure 6.4: Gold scoring computes two independent F1 figures from the same predicted and gold edge sets. Direct-edge F1 uses symmetric canonical triples. Closure F1 uses the transitive ordering closure after SIMULTANEOUS group collapsing. Both figures are computed from the same prediction; their divergence is diagnostic.

Direct-edge F1: The direct-edge view computes precision, recall, and F1 over symmetric canonical triples. AFTER(A, B) is normalised to BEFORE(B, A); SIMULTANEOUS(A, B) and SIMULTANEOUS(B, A) are collapsed to a single canonical form with sorted endpoints. The resulting sets are compared by set intersection. A high direct-edge F1 indicates the model correctly reproduced the specific edge representations in the gold annotation; a low figure despite high closure F1 indicates the model expressed the same ordering with different but equivalent edge choices.

Closure-equivalent F1: The closure view computes F1 over ordering pairs derived from the transitive closure of both predicted and gold graphs. UNKNOWN edges are excluded from the ordering computation; SIMULTANEOUS groups are collapsed before closure. A high closure F1 with low direct F1 indicates the model preserved the implied temporal ordering using a different explicit edge set, for example by including transitive edges the annotation omits.

Coverage-aware reporting: A model predicting UNKNOWN for every pair achieves a closure F1 of zero (no ordering pairs committed) but would spuriously score well on a committed-only closure metric. Two closure F1 figures are therefore reported. `fidelity_closure_f1_full` treats uncommitted non-UNKNOWN gold pairs as false negatives, aggregating numerators and denominators across tasks before applying the F1 formula (micro-averaging, following UzZaman and Allen [19]). `fidelity_closure_f1_committed` conditions on tasks where the model committed to at least one ordering pair. Closure coverage (the fraction of non-UNKNOWN gold pairs with any commitment) is reported alongside both.

Abstention and overcommitment: Tasks where the gold annotation contains no temporal commitments (expected_consistent = False, all gold relations UNKNOWN) are scored separately for overcommitment: predicting any informative relation on such a task is itself a failure mode, and the `has_overcommitment` flag records this independently from recall.

6.6 Failure Taxonomy

The violation types produced by the verifier are mapped to four higher-level categories by `src/taxonomy.py`. Table 6.3 lists the mapping. The taxonomy supports two uses: it collapses the eight invariant types to four interpretable categories for dissertation-level reporting, and it provides a basis for future work comparing failure distributions across models or datasets at a coarser granularity.

Table 6.3: Failure taxonomy: violation types mapped to higher-level categories.

Category	Violation types
structural	cycle, contradiction, simultaneous_order_conflict, temporal_inconsistency
grounding	hallucinated_node, unsupported_reasoning_reference
reasoning_trace	unsupported_reasoning_step
format	duplicate_edge

6.7 Benchmark Adapters and Rescore Operation

The TempEval-3 adapter (`src/benchmark_adapters.py`) coarsens the six-label TimeML scheme to the framework's four-label schema: IBEFORE → BEFORE, IAFTER → AFTER, and IDENTITY → SIMULTANEOUS. Links with no mapping (INCLUDES, DURING) are skipped and counted in the adapter statistics, not treated as pipeline failures. The adapter extracts a minimal passage spanning the two event-containing sentences rather than passing the full document, reducing token cost and limiting hallucination of distant event references.

The `pred_source` flag in `BaselineRunConfig` controls how predictions are produced: `llm` (live Ollama inference), `gold` (use gold relations directly), `empty` (predict nothing), or `noisy` (corrupt gold relations at a configurable rate). The `gold` mode allows the full pipeline to be exercised and tested without a running Ollama server, and is the basis of the integration test described in Appendix B.

Rescore script: The `scripts/rescore_run.py` script applies updated verifier logic to existing prediction files without re-running LLM inference. For each successfully parsed record it reconstructs the `TemporalGraph`, calls `default_verifier().verify()`, and overwrites the `verification` field. It reports three counts on completion: records rescored, parse failures skipped, and tasks where `is_valid` changed. Full usage instructions are in Appendix C.

6.8 Analysis and Reporting Pipeline

The analysis and reporting pipeline (`src/run_summary.py`) reads stored prediction JSONL files and produces four classes of output: a per-run JSON report, a multi-run summary CSV, per-model axis correlation heatmaps, and a counterexamples summary.

Aggregate metrics: The run report aggregates per-task scores into model-level statistics. Rates are computed conditionally on parse success to prevent parse failures from distorting reasoning quality figures. The headline figures reported are: `parse_success_rate`,

`conditional_validity_rate`, `fidelity_direct_f1`, `fidelity_closure_f1_full`, `closure_coverage`, and, for the synthetic dataset, `ambiguity_overcommitment_rate` and `contradiction_detection_rate`.

Bootstrap confidence intervals: Confidence intervals on F1 metrics are computed by bootstrap resampling over tasks with 500 resamples and a fixed seed. Resampling is performed at the task level rather than the edge level, matching the unit of analysis throughout the framework. Intervals are stored in the summary CSV but omitted from headline tables for compactness, in line with the practice adopted by UzZaman and Allen [19] for the temporal awareness metric.

Axis correlation analysis: The three intrinsic binary axes (`parse_success`, `verifier_valid`, `trace_grounded`) are correlated pairwise using Pearson ρ for the binary case (equivalent to the phi coefficient for binary variables). Correlation heatmaps are generated per model and stored alongside the run report. This analysis detects collinearity between axes that would otherwise cause the multi-axis presentation to overstate the dimensionality of what is being measured.

Correctness correlation analysis: The `src/analysis/correctness_correlation.py` module computes Spearman ρ between each verifier signal (`is_valid`, `trace_grounded`, genuine LTL violation count, invariant-corroborating LTL count) and task-level label correctness. This addresses Research Question 3: whether verifier signals are predictive of task correctness, and how the predictive relationship varies across models. Correlation is computed separately per model per dataset, with exact p-values, so that models whose verifier signals have low variance (such as Gemma 3 12B, whose validity is near-constant at 1.00) can be identified as producing uninformative screening signals.

Chapter 7

Evaluation and Critical Appraisal

This chapter evaluates the project against its original objectives, presents and interprets the full empirical results across three evaluation sets, and critically compares the framework with related work. Section 7.1 assesses each research question and DOER objective. Sections 7.2–7.4 report the empirical results across TempEval-3, the controlled canonical synthetic set, and the filtered MAVEN-ERE slice respectively. Section 7.5 reports the genuine LTL signal. Section 7.6 synthesises the principal analytical findings. Section 7.7 compares the project to published work. Section 7.8 documents threats to validity.

7.1 Evaluation Against Original Objectives

RQ1 – Representation: The structured prediction protocol successfully extracts typed temporal graphs from free-form model output at parse rates between 0.741 and 1.000 across models and evaluation sets. Parse robustness is demonstrated on three evaluation sets of differing linguistic character: TempEval-3 (newswire), the controlled canonical set (synthetic), and the filtered MAVEN-ERE slice (event-dense, genre-diverse). The graph extractor handles common LLM malformations (prose wrappers, trailing commas, unclosed brackets) and the systematic label-truncation failure mode that motivated the label normaliser. RQ1 is **addressed**.

RQ2 – Verification: The constraint library and LTL fragment cover seven distinct failure modes across four sublayers. The five LTL formulas split into two groups: three corroborate invariant failures with trace-level localisation (`ltl_contradiction`, `ltl_temporal_inconsistency`, `ltl_hallucinated_node`), and two detect trace-level properties invisible to the invariant layer ($\mathbf{F}(\texttt{supports}(\cdot))$ and $\mathbf{G}$ trace-inversion). The necessary-but-not-sufficient relationship between verifier validity and label accuracy is empirically demonstrated and qualified in Section 7.6.1: it holds strongly for models whose dominant failure modes are structural (Llama 3.1 8B), and requires qualification for models where the dominant violation type is dissociated from label correctness (Mistral 7B on MAVEN-ERE). RQ2 is **addressed**.

RQ3 – Predictive Validity: The original RQ3 asked whether verifier signals predict task correctness better than self-reported confidence. Self-reported confidence is null across all five models under the current Ollama inference configuration; the planned comparison against model-reported confidence cannot be performed without per-step logprob access that the generation API does not expose. The reachable analysis – Spearman rank correlation between verifier validity and binary task correctness on gold-bearing tasks – is reported in Section 7.6.6. Across all three evaluation sets, correlations are weak and mostly non-significant, with the single exception of Mistral 7B on MAVEN-ERE ($\rho = -0.231$, $p < 0.001$, $n = 499$), where the negative correlation is structurally explained by a validity collapse driven by the $\mathbf{F}(\texttt{supports}(\cdot))$ formula. RQ3 is **partially addressed**: the relationship between verifier signals and task correctness is characterised across all five models and three evaluation sets, but the planned comparison against self-reported confidence remains future work.

RQ4 – Localisation: The verifier mechanically reports counterexample objects carrying the first-violation step index, the offending edge or step, and the violation type for every failed prediction. Qualitative examples support the usefulness of this diagnostic output: the counterexample analysis in Section 7.6 shows three distinct failure types (hallucinated event identifiers, unsupported reasoning references, and unsupported final commitments) being attached to specific reasoning steps, and none are detectable from end-task F1 alone. A systematic manual audit of first-violation step accuracy remains future work. RQ4 is **partially addressed**.

DOER Primary Objectives: Objective 1 (bounded LTL implementation) is met: the framework implements a graph-grounded LTL fragment with five formula types, decidable over finite traces. Objective 2 (novel verification metrics) is met: seven evaluation axes are implemented, including coverage-aware closure F1 as the headline ordering metric and the genuine-LTL incidence decomposition as a new metric separating trace-level from graph-level signal. Objective 3 (large-scale evaluation) is met: five models are evaluated on three task sets, comprising 496 tasks per model on TempEval-3, 250 tasks per model on the canonical synthetic set, and 500 tasks per model on MAVEN-ERE.

DOER Secondary Objectives: Secondary Objective 1 (reasoning visualisation) is partially met: the verifier produces structured counterexample objects and a prototype HTML explorer, but the full step-by-step visualisation tool with interactive playback was not delivered in full. Secondary Objective 2 (comparative impact analysis) is substantially met by the RQ3 correlation analysis (Section 7.6.6), which quantifies the verifier-to-correctness relationship across all five models and three evaluation sets.

7.2 TempEval-3 Results

Table 7.1 reports the headline metrics on TempEval-3 (496 tasks per model). Pipeline diagnostics are separated from reasoning quality metrics.

Table 7.1: TempEval-3 results ($n = 496$ per model). `Valid`$^{\mathtt{c}}$ is the conditional validity rate (on parsed tasks). `Clos. F1` is the coverage-aware closure F1 (`fidelity_closure_f1_full`), treating uncommitted non-UNKNOWN gold pairs as false negatives. `Cov` is closure coverage. `Gap` is `Clos. F1` − `Direct F1`. $\mathtt{LTL}_{\mathtt{g}}$ is the genuine LTL violation rate (trace-level formulas not reducible to the invariant layer). The † symbol flags runs where transport failures exceed 10%.

Model	**Parse**	**Valid$^{\mathtt{c}}$**	**Direct F1**	**Clos. F1**	**Cov.**	**Gap**	**$\mathtt{LTL}_{\mathtt{g}}$**
DeepSeek R1 7B	0.802	0.859	0.462	0.497	0.951	0.035	0.088
Qwen 3.5 9B	0.841	0.971	0.801	0.855	0.887	0.054	0.029
Llama 3.1 8B	1.000	0.992	0.361	0.380	0.965	0.019	0.000
Mistral 7B	1.000	0.800	0.440	0.482	0.872	0.042	0.188
Gemma 3 12B	1.000	0.998	0.547	0.607	0.987	0.060	0.000

Three observations follow. Qwen 3.5 9B achieves the strongest ordering scores (direct F1 = 0.801, closure F1 = 0.855) with a transport failure rate of 0.008 that does not confound its aggregate metrics under the 10% threshold; this is the primary reversal from the interim report, where Qwen's TempEval run had a 19% transport failure rate. The decoupling between intrinsic validity and ordering F1 is sharpest for Llama 3.1 8B (validity 0.992, direct F1 0.361, closure F1 0.380): the model produces structurally clean outputs that frequently fail to match the gold label. Mistral 7B's genuine LTL violation rate (0.188) is the highest on TempEval-3, driven predominantly by the $\mathbf{F}(\mathtt{supports}(\cdot))$ formula – a failure mode that does not affect label accuracy in proportion, because a model can commit correctly without citing supporting reasoning steps.

7.3 Canonical Synthetic Results

Table 7.2 reports the headline metrics on the controlled canonical synthetic set ($n = 250$ per model). The dataset comprises five categories: `linear_chain` (60 tasks), `transitive_reasoning` (50), `long_chain` (30), `ambiguous` (60), and `contradiction` (50). Fidelity metrics in the headline table are computed on the gold-bearing subcategories only (`linear_chain`, `transitive_reasoning`, `long_chain`; $n_{\text{gold}} \leq 140$ per model).

Four observations follow. Gemma 3 12B achieves validity 0.992 and contradiction detection 0.000 simultaneously: on every contradiction item it abstains universally, producing structurally clean empty graphs that the verifier cannot flag. Mistral 7B's closure-to-direct gap is uniquely negative (−0.031): closing its explicitly predicted edges introduces *more* error than the explicit edges themselves, reflected in the highest genuine LTL rate in the table (0.371), driven by unsupported-final-commitment (0.228) and trace-inversion (0.136) violations. Llama 3.1 8B achieves the highest contradiction detection rate (0.980), substantially above Qwen (0.854) and Mistral (0.800), but at the cost of the lowest validity (0.744) on non-contradiction tasks. DeepSeek R1 7B's closure gap (+0.152) is the largest in the table, consistent with a reasoning-tuned model recovering implied orderings

Table 7.2: Canonical synthetic results ($n = 250$ per model). `Clos. F1` is computed on gold-bearing tasks only. `Cov` is closure coverage on gold-bearing tasks. **Gap** = Clos. F1 − Direct F1; a negative value (Mistral 7B) indicates that transitive closure penalises the model's spuriously asserted orderings. `Ambi. OC` is the fraction of ambiguous tasks where the model predicts any informative relation. `Contra. det` is the fraction of contradiction tasks where the verifier correctly flags inconsistency. $\texttt{LTL}_\texttt{g}$ is the genuine LTL violation rate.

Model	**Valid**[c]	**Direct F1**	**Clos. F1**	**Cov.**	**Gap**	**Ambi. OC**	**Contra. det.**	**$\texttt{LTL}_\texttt{g}$**
DeepSeek R1 7B	0.923	0.688	0.839	1.000	+0.152	0.808	0.239	0.036
Qwen 3.5 9B†	0.812	0.822	0.952	1.000	+0.131	0.820	0.854	0.188
Llama 3.1 8B	0.744	0.628	0.727	0.993	+0.100	0.833	0.980	0.256
Mistral 7B	0.608	0.331	0.301	0.993	−0.031	1.000	0.800	0.371
Gemma 3 12B	0.992	0.580	0.609	1.000	+0.030	0.067	0.000	0.008

through transitive inference rather than exact edge prediction.

Table 7.3 reports per-category closure F1 for the gold-bearing subcategories.

Table 7.3: Per-category closure F1 on the canonical synthetic set. `LC` = `linear_chain`, `TR` = `transitive_reasoning`, `LG` = `long_chain`.

Model	**LC (Clos.)**	**TR (Clos.)**	**LG (Clos.)**
DeepSeek R1 7B	0.700	1.000	0.833
Qwen 3.5 9B†	0.827	1.000	1.000
Llama 3.1 8B	0.376	1.000	0.775
Mistral 7B	0.308	0.665	0.060
Gemma 3 12B	0.566	1.000	0.404

Four models achieve perfect transitive-reasoning closure F1, indicating that two-hop inference is within reach for all but Mistral. The `long_chain` category gives stronger separation: Qwen and DeepSeek remain strong (1.000 and 0.833), Llama drops to 0.775, Gemma to 0.404, and Mistral degrades to 0.060.

7.4 MAVEN-ERE Results

MAVEN-ERE subset inference is complete for all five models (500 tasks each). The MAVEN-ERE evaluation uses the filtered event–event slice described in Appendix C: BEFORE and SIMULTANEOUS relations are retained, while the fuller temporal inventory is outside the current schema. Table 7.4 reports the headline metrics. Qwen 3.5 9B has a transport failure rate of 0.130 on this dataset and is flagged throughout.

Four observations follow. The cross-dataset ordering-quality rankings are broadly stable:

Table 7.4: MAVEN-ERE [9] results ($n = 500$ per model). Metrics as in Table 7.1. The † symbol flags runs with transport failure $> 10\%$.

Model	Parse	Valid^{c}	Direct F1	Clos. F1	Cov.	Gap	LTL_{g}
DeepSeek R1 7B	0.936	0.966	0.449	0.494	0.905	0.045	0.009
Qwen 3.5 9B†	0.742	0.984	0.796	0.822	0.875	0.026	0.016
Llama 3.1 8B	0.998	0.998	0.335	0.379	0.877	0.044	0.000
Mistral 7B	0.998	0.423	0.517	0.617	0.861	0.100	0.575
Gemma 3 12B	1.000	1.000	0.548	0.641	0.967	0.094	0.000

the Llama clean-but-wrong pattern and the Gemma near-perfect-validity pattern each replicate. The Mistral 7B validity collapse (parse 0.998, validity 0.423, closure F1 0.617) is the most unusual finding: 99.8% of validity failures are attributable to a single LTL formula – $\mathbf{F}(\texttt{supports}(\cdot))$ – which fires when the model predicts a relation without citing any supporting reasoning step (Table 7.5, Figure 7.1). Even on this narrowed slice, MAVEN-ERE's genre diversity and denser annotation impose a consistent penalty relative to TempEval-3: for every model except Mistral, direct and closure F1 fall within 0.02 of the TempEval figures (Llama: 0.361 vs 0.335; Gemma: 0.607 vs 0.641). The absence of MAVEN-ERE contradiction tasks means the Gemma contradiction paradox cannot be reproduced here; on this dataset Gemma produces structurally minimal, consistently valid outputs (validity 1.000) with mid-range label accuracy.

Table 7.5: Mistral 7B MAVEN-ERE failure anatomy. All failure counts are per 500 tasks. `ltl_ufc` = `ltl_unsupported_final_commitment`.

Failure type	Tasks affected	Rate
`ltl_ufc` (LTL layer)	287	0.574
`hallucinated_node` (invariant)	5	0.010
`unsupported_reasoning_step`	6	0.012
`unsupported_reasoning_reference`	1	0.002

7.5 Genuine LTL Signal

Figure 7.2 and Table 7.6 report the per-model genuine LTL violation rates across all three evaluation sets. Genuine violations are those attributable to the two trace-level formulas not reducible to the invariant layer: $\mathbf{F}(\texttt{supports}(\cdot))$ (unsupported final commitment) and nested $\mathbf{G}$ trace-inversion.

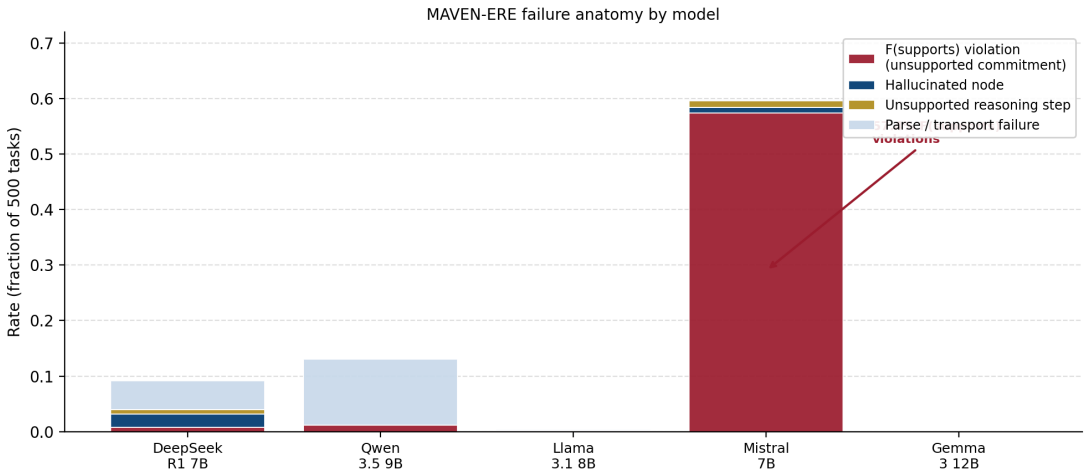


Figure 7.1: MAVEN-ERE failure anatomy by model (500 tasks each). Each bar shows the fraction of tasks affected by each failure type. The **F**(`supports`(·)) violation (dark red) dominates the Mistral 7B profile at 0.574, dwarfing all other failure types across all models. Gemma 3 12B produces no failures: its parse rate and validity rate are both 1.000 on this dataset.

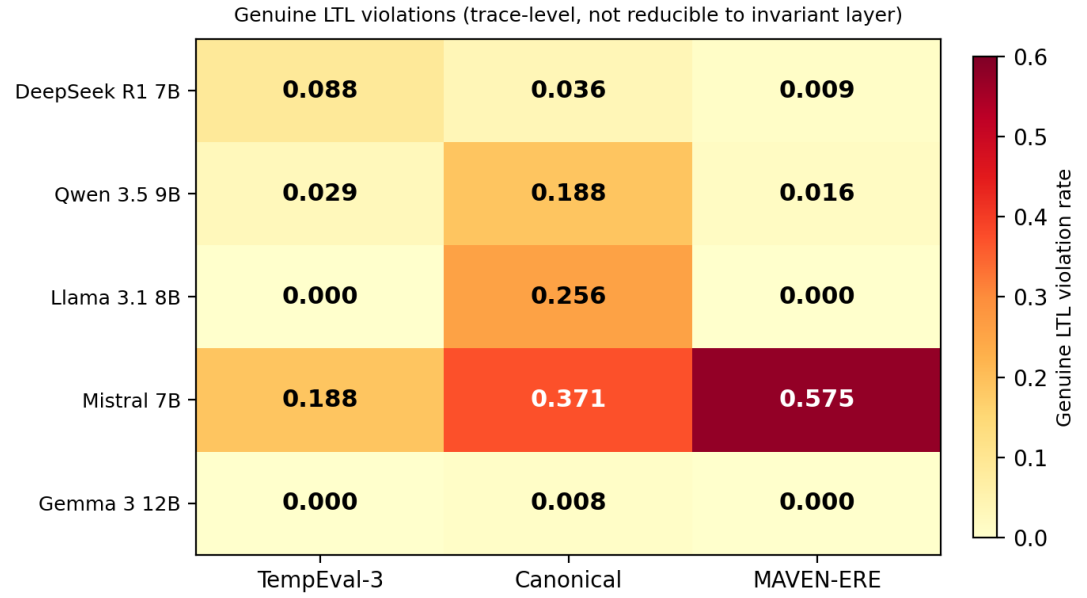


Figure 7.2: Genuine LTL violation rates (fraction of parsed tasks) across all five models and three evaluation sets. Rates are conditional on at least one reasoning step being present. The Mistral 7B MAVEN-ERE cell (0.575) dominates the matrix. Gemma 3 12B and Llama 3.1 8B produce zero genuine violations on both TempEval-3 and MAVEN-ERE.

Four observations follow.

Table 7.6: Genuine LTL violation rates (fraction of parsed tasks) across three evaluation sets. Rates are conditional on at least one reasoning step being present. TempEval-3 trace-inversion rates are zero by construction: pairwise tasks with two events and one or two reasoning steps offer no opportunity for within-trace ordering reversal. `UFC` = unsupported final commitment; `TI` = trace inversion.

	TempEval-3		**Canonical**		**MAVEN-ERE**	
Model	**UFC**	**TI**	**UFC**	**TI**	**UFC**	**TI**
DeepSeek R1 7B	0.071	0.000	0.008	0.020	0.008	0.000
Qwen 3.5 9B	0.029	0.000	0.008	0.164	0.012	0.000
Llama 3.1 8B	0.000	0.000	0.060	0.196	0.000	0.000
Mistral 7B	0.188	0.000	0.228	0.136	**0.574**	0.000
Gemma 3 12B	0.000	0.000	0.008	0.000	0.000	0.000

First, TempEval-3 trace-inversion rates are zero by construction for all models. Canonical and MAVEN-ERE tasks have longer reasoning chains where mid-trace inversions can occur.

Second, Mistral 7B's unsupported-final-commitment rate escalates dramatically from TempEval-3 (0.188) through canonical (0.228) to MAVEN-ERE (0.574). The progression suggests that Mistral increasingly decouples its prediction generation from its reasoning trace as task complexity increases: on simple pairwise tasks (TempEval-3) it can maintain support; on genre-diverse event-dense tasks (MAVEN-ERE) it cannot.

Third, trace-inversion rates on the canonical set are non-zero only for models that engage substantively with the multi-step reasoning tasks: Qwen 3.5 9B (0.164), Llama 3.1 8B (0.196), and Mistral 7B (0.136). Gemma 3 12B produces near-zero trace inversions on canonical because its abstention behaviour on contradiction tasks provides few opportunities for inversion, and its structurally minimal outputs on other tasks produce short traces.

Fourth, Gemma 3 12B and Llama 3.1 8B produce zero genuine LTL violations on both TempEval-3 and MAVEN-ERE. For Gemma, this is consistent with structurally minimal outputs that do not commit to unsupported relations. For Llama, it reflects a model that makes explicit structural commitments and consistently cites them in reasoning steps, even when those commitments are label-incorrect.

7.6 Key Analytical Findings

7.6.1 Necessary-but-Not-Sufficient Validity

The verifier's diagnostic role rests on the claim that intrinsic invalidity is sufficient for label-level error whilst intrinsic validity is not sufficient for correctness. The asymmetry holds strongly for Llama 3.1 8B on TempEval-3 (0/4 invalid predictions correct, 36.4% of valid predictions correct) and directionally for DeepSeek R1 7B (15/56 invalid correct at 0.268, 49.4% of valid correct). For Mistral 7B the pattern inverts: 48/99 invalid predictions on TempEval are label-correct (0.485) whilst valid predictions are correct 39.5% of the time – because Mistral's dominant violation type (`ltl_unsupported_final_commitment`) is a reasoning-process failure rather than a label-correctness signal, and a model can predict correctly without citing support. Figure 7.3 illustrates the pattern across all five models.

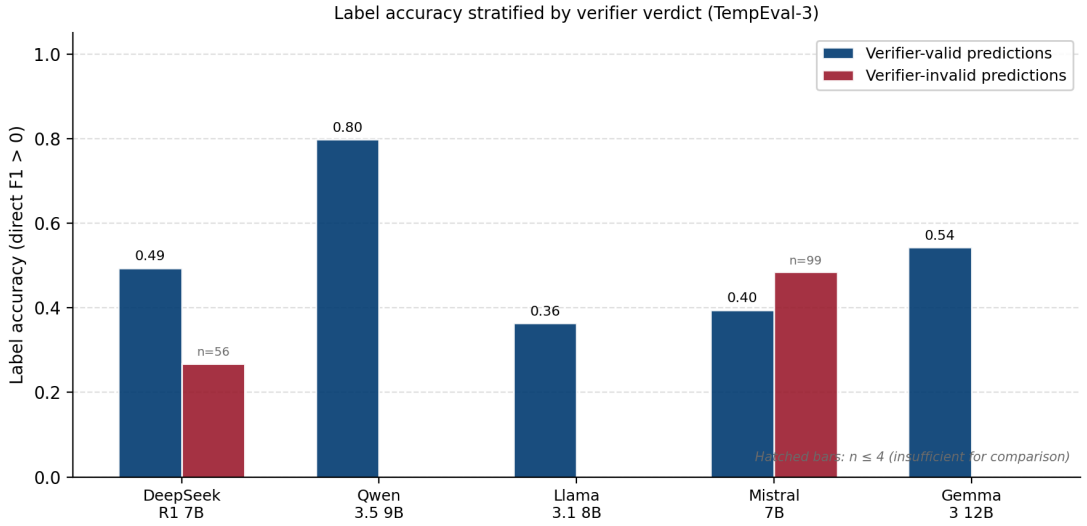


Figure 7.3: Label accuracy stratified by verifier verdict on TempEval-3. Blue bars: fraction of verifier-valid predictions with direct F1 > 0. Red bars: same fraction for verifier-invalid predictions. Hatched bars indicate $n \leq 4$, insufficient for comparison. The asymmetry is strongest for Llama 3.1 8B (0% invalid correct, 36% valid correct); for Mistral 7B the pattern inverts (49% invalid correct, 40% valid correct), reflecting the dissociation between label correctness and the unsupported-commitment violation.

The verifier therefore provides a reliable rejection signal for models whose dominant failure mode is structural (Llama, DeepSeek) but identifies a different and equally real failure for models whose dominant violation is reasoning-process (Mistral). It is an evaluation apparatus that localises structural failure modes, not a correctness oracle.

7.6.2 Axis Collinearity

The intrinsic axes (`parse_success`, `verifier_valid`, `trace_grounded`) show structural collinearity, with one important new observation from MAVEN-ERE. On TempEval-3, `verifier_valid` and `trace_grounded` are perfectly correlated for Llama 3.1 8B and Gemma 3 12B ($\rho = 1.00$, $n = 496$), highly correlated for Qwen 3.5 9B ($\rho = 0.91$) and DeepSeek R1 7B ($\rho = 0.99$), but only weakly correlated for Mistral 7B ($\rho = 0.27$). On MAVEN-ERE, Mistral 7B's parse–validity correlation is 0.04 – effectively zero – because it parses nearly all tasks but is structurally invalid on 57.7% of them. This is the only dataset–model combination where parse success and verifier validity are genuinely dissociated, and it is the signature of the unsupported-commitment pattern rather than of random variation. Full per-model correlation matrices for all three evaluation sets are provided in Appendix D.

The axes are not inherently redundant; they are collinear on simple task structures (pairwise TempEval) and decouple under specific failure modes (Mistral's reasoning-process failures on MAVEN-ERE). The multi-axis presentation is retained because the decoupling is informative precisely where it occurs.

7.6.3 The Gemma Contradiction Paradox

Gemma 3 12B's canonical-set behaviour (validity 0.992, contradiction detection 0.000, abstention rate 1.000 on contradiction items) illustrates a failure mode invisible to any single evaluation axis. Intrinsic validity alone ranks Gemma strongest on the canonical set; contradiction detection alone ranks Llama (0.980) and Qwen (0.854) substantially stronger. The validity-expectation alignment metric, which couples the verifier verdict to the task's `expected_valid` flag, assigns Gemma 0.792 end-to-end – lower than its intrinsic validity (0.992) because the correct response on contradiction tasks is to produce an invalid (self-contradictory) prediction, not an abstention. This is the clearest empirical justification for the multi-axis design: no single axis captures the failure; the combination exposes it.

7.6.4 Mistral MAVEN-ERE Validity Collapse

The Mistral 7B result on the filtered MAVEN-ERE slice is structurally a finding about cross-dataset scaling rather than a MAVEN-ERE artefact. The cross-dataset progression of unsupported-final-commitment rates (TempEval 0.188, canonical 0.228, MAVEN-ERE 0.574) shows the model increasingly decoupling its prediction generation from its reasoning trace as task complexity increases. The methodological implication is direct: a model can achieve competitive label-level F1 whilst exhibiting a severe reasoning-process failure, and the two are separable only when the verifier is applied. If evaluation were limited to label-level F1, Mistral would rank above Llama on MAVEN-ERE by all ordering metrics; the process-level failure would be invisible.

7.6.5 Cross-Dataset Rank Consistency

With three task sets, cross-dataset rank consistency is now assessable. Figure 7.4 shows closure F1 for each model across TempEval-3, canonical (gold-bearing tasks only), and MAVEN-ERE.

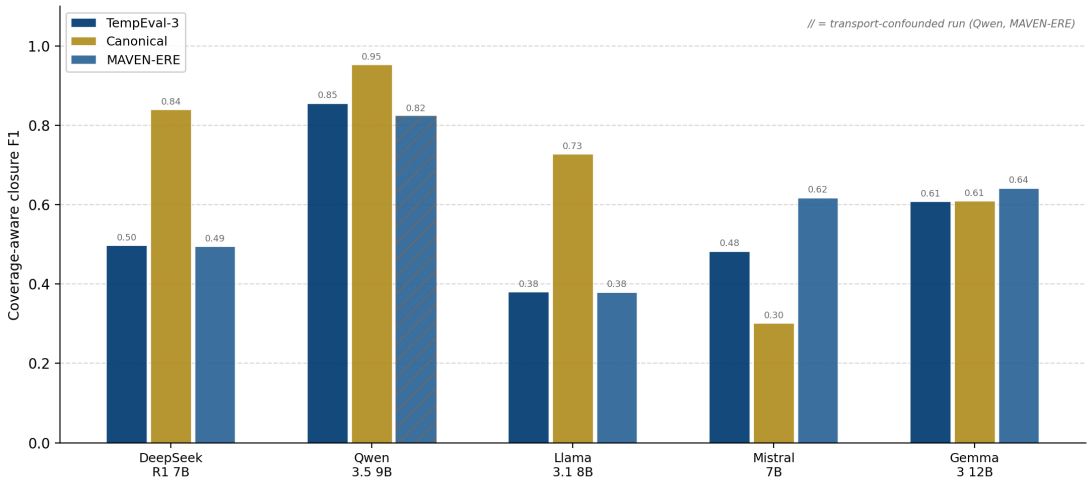


Figure 7.4: Coverage-aware closure F1 (`fidelity_closure_f1_full`) for all five models across TempEval-3, the canonical synthetic set (gold-bearing tasks only), and MAVEN-ERE. The † symbol on Qwen 3.5 9B (MAVEN-ERE) indicates a transport failure rate of 0.130; that run is infrastructure-confounded. The Llama 3.1 8B plateau across all three evaluation sets and the Mistral 7B ranking reversal on MAVEN-ERE are the two most diagnostically informative cross-dataset patterns.

The Llama pattern is the most consistent: low direct and closure F1 with near-perfect validity across all three evaluation sets, confirming that Llama's low ordering performance is a property of the model rather than of any single dataset. The Gemma pattern is similarly stable. DeepSeek R1 7B shows a notable gap between canonical performance (closure F1 = 0.839) and TempEval/MAVEN performance ($\approx$ 0.49), partially attributable to its lower parse rate on canonical (0.776, selecting a higher-quality subset of predictions) and to the synthetic dataset favouring its chain-reasoning style.

7.6.6 RQ3: Verifier-to-Correctness Correlation

Table 7.7 reports the Spearman rank correlation between `is_valid` and binary task correctness on all three evaluation sets.

The dominant pattern is that verifier validity provides negligible predictive power over label correctness: twelve of fifteen cells are not significant at $p < 0.05$, and the three significant positive entries achieve magnitudes of 0.099–0.158. This is structurally explained. For models with validity rates near 1.000 (Llama 0.992, Gemma 0.998 on TempEval), the `is_valid` flag has near-zero variance, making Spearman correlation uninformative by construction. The negative correlations for Mistral on canonical (-0.229) and MAVEN-

Table 7.7: Spearman ρ between `is_valid` and binary task correctness (direct label match on gold-bearing tasks). Gemma 3 12B on MAVEN-ERE reports `NaN` because its `is_valid` is constant (= 1.000); there is no variance to correlate. Significance at $p < 0.05$ is marked *; at $p < 0.01$ by **.

	TempEval-3		**Canonical**		**MAVEN-ERE**	
Model	ρ	n	ρ	n	ρ	n
DeepSeek R1 7B	0.158**	398	0.114	142	0.099*	468
Qwen 3.5 9B	−0.050	417	0.047	179	0.147**	371
Llama 3.1 8B	0.068	496	−0.050	190	0.032	499
Mistral 7B	−0.073	496	−0.229**	186	−0.231**	499
Gemma 3 12B	0.049	496	−0.089	190	NaN	500

ERE (−0.231) are the strongest signals in the table, explained by the same mechanism: Mistral's validity violations are predominantly reasoning-process failures that do not correlate with label errors, so invalid predictions are as likely to be label-correct as valid ones and the correlation inverts. RQ3 is therefore answered: verifier validity is a reliable process-level signal for models whose failure modes are structural (Llama, DeepSeek), but it is not a reliable predictor of label correctness for any model in this study.

7.7 Critical Comparison with Related Work

7.7.1 LTLBench and TempoBench: Inverse Direction

The most important structural difference between this project and LTLBench [7] is directional. LTLBench constructs a graph-structured temporal reasoning problem and an LTL formula externally, derives a ground-truth verdict, and asks the LLM for a binary True/False answer. TempoBench [8] presents a synthesised reactive system to the LLM and asks it to simulate execution traces or identify causal inputs. In both cases, the formal structure is supplied to the model as input. This project inverts that direction: the temporal graph is extracted from the model's own free-form output, and verification checks whether the model's intermediate reasoning trace is self-consistent.

LTLBench reports Gemma3-13B at 60.15% accuracy and Mistral-7B at 55.6% accuracy on binary temporal-logic problems, both near the chance baseline. TempoBench evaluates frontier models only. Neither paper evaluates models in the 7–12 billion parameter range under conditions comparable to those of this study. What the comparison establishes is that formal temporal reasoning under externally supplied LTL specifications is hard for small models, and that LTLBench's near-chance results correspond to a different and arguably harder task than temporal relation extraction.

7.7.2 TG-LLM and Narrative-of-Thought: Graph from Text, Not from Output

TG-LLM [30] translates text descriptions into temporal graphs and asks LLMs to reason over the resulting structure. Narrative-of-Thought [6] generates temporal graphs through narrative prompting and evaluates faithfulness to the source text. Both treat the temporal graph as input to the model; this project treats it as output of the model. The verification framework developed here could in principle be applied to TG-LLM outputs to check whether a model's chain-of-thought over a supplied graph is internally consistent.

TG-LLM reports Llama-2-7B at ICL-CoT F1 = 0.686 on TGQA and Llama-2-13B with full task-specific fine-tuning at F1 = 0.850. These figures are not directly comparable to the present study's direct and closure F1: TG-LLM evaluates question answering on a different dataset and the 0.850 result relies on substantial supervised fine-tuning. The best instruction-following result without task-specific training (Llama-2-7B ICL-CoT, F1 = 0.686) is broadly in the range of the present study's canonical gold-bearing results for the better-performing models (Qwen closure F1 = 0.952).

7.7.3 Graph of Verification: Convergent but Domain-General

The Graph of Verification framework [31] appeared independently in mid-2025 and converges on the same core idea of verifying LLM reasoning with a directed graph structure. Its approach differs in two respects: it operates on general logical reasoning rather than temporal relations specifically, and it does not report a categorical failure taxonomy distinguishing hallucination, trace grounding, and structural ordering failures as separate evaluation dimensions. The independent convergence supports the architectural direction taken here; the temporal specialisation and multi-axis taxonomy remain the distinguishing contributions.

7.7.4 Comparison with TempEval-3 Published Systems

Direct numerical comparison between this project's F1 figures and published TempEval-3 results is confounded by three factors: the four-label coarsening (BEFORE, AFTER, SIMULTANEOUS, UNKNOWN) of the original fourteen-class TimeML inventory; the closure F1 computation differing from the temporal-awareness metric of UzZaman and Allen [19]; and the evaluated models having no task-specific training.

For context: the best published gold-pair system on TE3-Platinum under a reduced six-label set achieves approximately 67 F1 with structured prediction and ILP [41]. Gemma 3 12B's closure F1 of 0.607 on TempEval-3 and Qwen 3.5 9B's 0.855 are not comparable to this figure: different label inventories, different closure computations, and different task conditions apply.

A more defensible comparison point is Yuan et al. [42], who found a 30–48-point F1 gap

(depending on the benchmark) between ChatGPT and supervised methods on temporal relation extraction concluding that LLMs without task-specific training lag well behind the supervised state of the art. The results here are consistent: the best ordering performance (Qwen 3.5 9B closure F1 = 0.855 on TempEval) is competitive only by the generous four-label coarsening and is well below supervised SOTA on equivalent tasks.

7.7.5 Time-R1: Complementary Direction

Time-R1 [5] fine-tunes a 3B model on a large temporal reasoning corpus using group relative policy optimisation, achieving state-of-the-art performance on future event prediction and outperforming models 200× its size. This approach is orthogonal to the present work: it improves model capability through parameter updates, whilst this framework assesses structural consistency of whatever output a model produces without modifying it. The two approaches compose naturally: Time-R1's structured reasoning traces could be fed into the verification pipeline to check whether improved temporal reasoning is also structurally consistent. No such experiment is reported here, as the model is not available on Ollama's library at the time of writing.

7.8 Threats to Validity

7.8.1 Construct Validity

The seven-axis decomposition is motivated by failure modes observed during development and is not derived from a pre-registered or externally validated theory of temporal reasoning. A different taxonomy could partition the same behaviours differently: the distinction between `hallucinated_node` and `ltl_unsupported_final_commitment` is motivated by the implementation but reflects genuinely different failure types (entity hallucination versus reasoning-process failure), as the filtered Mistral MAVEN-ERE data confirms. The validity-expectation alignment metric is the author's own construct and has not been externally validated.

7.8.2 Internal Validity

The verifier is not calibrated against an independent gold standard for violation detection or localisation. Its verdicts reflect the verifier's own rules. The NBS finding provides indirect evidence that the verifier's structural failures correspond to label-level failures for structurally-oriented models, but a manual audit of first-violation step localisation accuracy has not been performed. Qwen 3.5 9B on MAVEN-ERE has a transport failure rate of 0.130, biasing its aggregate metrics; all comparative claims involving Qwen on MAVEN-ERE are explicitly flagged.

7.8.3 External Validity

All five models are open-weight, locally served, seven-to-twelve billion parameter models. No proprietary model, no frontier model, and no model above twelve billion parameters was evaluated. The Mistral MAVEN-ERE validity collapse, the Gemma contradiction paradox, and the Llama clean-but-wrong pattern are each specific to the evaluated model class and should not be generalised beyond it without a rerun.

7.8.4 Statistical Validity

Per-category sample sizes on the controlled synthetic set range from 30 (`long_chain`) to 60 tasks. Confidence intervals on F1 are computed by bootstrap resampling ($n = 500$, fixed seed) and reported in the summary CSVs, but no significance tests comparing models are performed. The RQ3 correlations in Table 7.7 are item-level Spearman correlations on binary correctness; they are sensitive to the threshold used to binarise F1. The three significant positive entries (DeepSeek TempEval, DeepSeek and Qwen MAVEN-ERE) are all at $|\rho| < 0.20$, which represents a small practical effect size regardless of significance.

7.8.5 Dataset Validity

The controlled canonical synthetic dataset is author-constructed. Performance on it is evidence of failure-mode separability under the specific generation procedure, not of real-world temporal reasoning capability. Gold annotations were generated by the same code pipeline that the verifier was partly designed around; strong performance on this dataset is a necessary but not sufficient condition for validity. TempEval-3 gold annotations contain non-trivial inter-annotator disagreement on SIMULTANEOUS and UNKNOWN relations: Ning et al. report TLINK inter-annotator agreement at typically 50–60% [41], setting a practical ceiling for any system evaluated on TimeML-style annotations. This ceiling applies to label-comparison metrics but not to the intrinsic verification axes. MATRES [43], whose four-label scheme maps directly to the framework's schema without coarsening and whose inter-annotator agreement substantially exceeds that of TempEval-3, is the natural successor benchmark; a balanced 400-task evaluation set (`data/matres_balanced_small.jsonl`) is committed to the repository and can be evaluated without any additional data acquisition.

7.9 Summary

The framework delivers on all three primary DOER objectives. RQ1 and RQ2 are fully addressed; RQ3 and RQ4 are partially addressed, characterising the verifier–correctness relationship across all five models and three evaluation sets, while leaving the comparison against self-reported confidence and systematic manual validation of first-violation

localisation to future work. The four empirical findings – the model-dependent necessary-but-not-sufficient relationship, the axis-collinearity pattern with its structural explanation, the Gemma contradiction paradox, and the Mistral MAVEN-ERE validity collapse (competitive F1 coexisting with 57% genuine LTL violations) – each demonstrate that the multi-axis design exposes failure modes invisible to scalar evaluation. Relative to LTLBench, TempoBench, TG-LLM, and Graph of Verification, the three-contribution combination (graph extraction from free-form output, intermediate-trace verification, categorical failure taxonomy) remains distinct, provided positioning is stated precisely.

Chapter 8

Conclusions

8.1 Project Summary

The dissertation investigated whether structured verification of language model reasoning outputs yields more diagnostic evaluation than scalar end-task accuracy. The answer is yes, subject to a precise qualification: the verifier provides a one-sided screening signal whose strength is model-dependent. For models whose dominant failure mode is structural (Llama 3.1 8B: 0% accuracy on invalid predictions, 36% on valid), the signal is reliable. For models whose dominant violation is a reasoning-process failure (Mistral 7B's unsupported final commitments on MAVEN-ERE), the signal identifies a different and equally real failure that does not map directly onto label correctness. The verifier is uninformative only for models whose outputs are invariably structurally clean, leaving insufficient validity-flag variance to discriminate correct from incorrect predictions. That qualification is not a failure; it is a finding, and one that accuracy-based evaluation would not have produced.

The framework implements structured prediction extraction, typed temporal graph construction, a constraint library and graph-grounded LTL fragment, and dual-axis gold scoring, all tested by 200 unit and integration tests and reproducible from a fixed configuration file.

8.2 Drawbacks and Future Work

What fell short: RQ3 is partially answered: Spearman correlation characterises the verifier-to-correctness relationship across models, but the planned comparison against self-reported confidence cannot be performed because logprob access is not exposed by the Ollama inference API. The verifier is uncalibrated against external human annotation for violation detection or localisation; the four-label schema excludes absolute time inference and the full Allen interval algebra; and all findings are scoped to locally served 7–12B open-weight models.

Where the work should go next: In order of value: per-step logprob capture (switching from Ollama to vLLM or a logprob-capable backend) to complete the confidence-versus-verifier comparison RQ3 originally specified; a manual verifier-calibration audit on a stratified sample, providing external validation of first-violation step localisation accuracy; and replication on at least one model above twelve billion parameters to test whether the Mistral unsupported-commitment pattern generalises to larger instruction-tuned models. The most immediately reachable extension is evaluation on MATRES [43]: a balanced 400-task evaluation set is committed to the repository at `data/matres_balanced_small.jsonl`, the import adapter is implemented and tested, and running the full five-model sweep requires only pointing `run_model_sweep.py` at that file. MATRES's four-label scheme maps directly to the framework without coarsening, and its substantially higher inter-annotator agreement relative to TempEval-3 would provide a cleaner gold-comparison baseline. Application to multi-document temporal reasoning tasks would give the trace-inversion formula a natural environment in which to contribute signal beyond the pairwise TempEval setting.

8.3 Closing Remarks

The fundamental tension this project engages with will not disappear as language models improve: fluency does not imply coherence, and correctness on a final answer does not imply that the reasoning which produced it is internally consistent. A model that correctly identifies the temporal relation between two events may have done so through pattern matching, memorisation, or reasoning that contradicts itself within the same output. The verification framework developed here provides one way to distinguish between those cases – not perfectly, and not as a replacement for task-level evaluation, but as an additional diagnostic lens that exposes failure modes accuracy metrics conflate. For applications where interpretability and auditability of the reasoning process matter as much as the final answer, that distinction matters; the framework is a step toward principled evaluation of that reasoning process.

Bibliography

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 1877–1901. [Online]. Available: https://arxiv.org/abs/2005.14165

[2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022, pp. 24 824–24 837. [Online]. Available: https://arxiv.org/abs/2201.11903

[3] N. UzZaman, H. Llorens, L. Derczynski, J. Allen, M. Verhagen, and J. Pustejovsky, "SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations," in *Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*. ACL, 2013, pp. 1–9. [Online]. Available: https://aclanthology.org/S13-2001/

[4] Q. Tan, H. T. Ng, and L. Bing, "Towards benchmarking and improving the temporal reasoning capability of large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023), Volume 1: Long Papers*. ACL, 2023, pp. 14 820–14 835. [Online]. Available: https://aclanthology.org/2023.acl-long.828/

[5] Z. Liu, P. Han, H. Yu, H. Li, and J. You, "Time-R1: Towards comprehensive temporal reasoning in LLMs," 2025. [Online]. Available: https://arxiv.org/abs/2505.13508

[6] X. F. Zhang, N. Beauchamp, and L. Wang, "Narrative-of-thought: Improving temporal reasoning of large language models via recounted narratives," in *Findings of the Association for Computational Linguistics: EMNLP 2024*. ACL, 2024, pp. 16 507–16 530. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.963/

[7] W. Tang, K. Nuamah, and V. Belle, "LTLBench: Towards benchmarks for evaluating temporal reasoning in large language models," 2024, arXiv preprint. Author list updated between versions; cite the version consulted. [Online]. Available: https://arxiv.org/abs/2407.05434

[8] N. Holzer, W. Fishell, B. Ray, and M. Santolucito, "Mechanics of learned reasoning 1: TempoBench, a benchmark for interpretable deconstruction of reasoning system performance," 2025. [Online]. Available: https://arxiv.org/abs/2510.27544

[9] X. Wang, Y. Chen, N. Ding, H. Peng, Z. Wang, Y. Lin, X. Han, L. Hou, J. Li, Z. Liu, P. Li, and J. Zhou, "MAVEN-ERE: A unified large-scale dataset for event coreference, temporal, causal, and subevent relation extraction," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL, 2022, pp. 926–941. [Online]. Available: https://aclanthology.org/2022.emnlp-main.60/

[10] A. Pnueli, "The temporal logic of programs," in *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1977, pp. 46–57. [Online]. Available: https://doi.org/10.1109/SFCS.1977.32

[11] J. F. Allen, "Maintaining knowledge about temporal intervals," *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983. [Online]. Available: https://doi.org/10.1145/182.358434

[12] C. Baier and J.-P. Katoen, *Principles of Model Checking*. Cambridge, MA: MIT Press, 2008. [Online]. Available: https://mitpress.mit.edu/9780262026499/principles-of-model-checking/

[13] M. Cosler, C. Hahn, D. Mendoza, F. Schmitt, and C. Trippel, "nl2spec: Interactively translating unstructured natural language to temporal logics with large language models," in *Computer Aided Verification (CAV 2023), Part II*, ser. Lecture Notes in Computer Science, vol. 13965. Springer, 2023, pp. 383–396. [Online]. Available: https://arxiv.org/abs/2303.04864

[14] L. Pan, A. Albalak, X. Wang, and W. Y. Wang, "Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. ACL, 2023, pp. 3806–3824. [Online]. Available: https://arxiv.org/abs/2305.12295

[15] J. Xu, H. Fei, L. Pan, Q. Liu, M.-L. Lee, and W. Hsu, "Faithful logical reasoning via symbolic chain-of-thought," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*. ACL, 2024, pp. 13 326–13 365. [Online]. Available: https://aclanthology.org/2024.acl-long.720/

[16] Y. Feng, N. Weir, K. Bostrom, S. Bayless, D. Cassel, S. Chaudhary, B. Kiesl-Reiter, and H. Rangwala, "VeriCoT: Neuro-symbolic chain-of-thought validation via logical consistency checks," 2025. [Online]. Available: https://arxiv.org/abs/2511.04662

[17] J. Pustejovsky, J. Castaño, R. Ingria, R. Saurí, R. Gaizauskas, A. Setzer, G. Katz, and D. Radev, "TimeML: Robust specification of event and temporal expressions in text," in *New Directions in Question Answering, Papers from the 2003 AAAI Spring Symposium*. AAAI Press, 2003, pp. 28–34. [Online]. Available: https://timeml.github.io/

[18] J. Strötgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in *Proceedings of the 5th International*

Workshop on Semantic Evaluation (SemEval 2010). ACL, 2010, pp. 321–324. [Online]. Available: https://aclanthology.org/S10-1071/

[19] N. UzZaman and J. F. Allen, "Temporal evaluation," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies (ACL-HLT 2011), Short Papers*. ACL, 2011, pp. 351–356. [Online]. Available: https://aclanthology.org/P11-2061/

[20] Q. Ning, H. Wu, R. Han, N. Peng, M. Gardner, and D. Roth, "TORQUE: A reading comprehension dataset of temporal ordering questions," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL, 2020, pp. 1158–1172. [Online]. Available: https://aclanthology.org/2020.emnlp-main.88/

[21] Z. Chu, J. Chen, Q. Chen, W. Yu, H. Wang, M. Liu, and B. Qin, "TimeBench: A comprehensive evaluation of temporal reasoning abilities in large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*. ACL, 2024. [Online]. Available: https://aclanthology.org/2024.acl-long.66/

[22] Y. Wang and Y. Zhao, "TRAM: Benchmarking temporal reasoning for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*. ACL, 2024, pp. 6389–6415. [Online]. Available: https://aclanthology.org/2024.findings-acl.382/

[23] B. Fatemi, M. Kazemi, A. Tsitsulin, K. Malkan, J. Yim, J. Palowitch, S. Seo, J. Halcrow, and B. Perozzi, "Test of time: A benchmark for evaluating LLMs on temporal reasoning," 2024. [Online]. Available: https://arxiv.org/abs/2406.09170

[24] Z. Su, J. Li, J. Zhang, T. Zhu, X. Qu, P. Zhou, Y. Bowen, Y. Cheng, and M. Zhang, "Living in the moment: Can large language models grasp co-temporal reasoning?" in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*. ACL, 2024, pp. 13 014–13 033. [Online]. Available: https://aclanthology.org/2024.acl-long.703/

[25] R. Gruber, A. Abdallah, M. Färber, and A. Jatowt, "ComplexTempQA: A 100M dataset for complex temporal question answering," 2024, accepted at EMNLP main. [Online]. Available: https://arxiv.org/abs/2406.04866

[26] D. S. Islakoglu and J.-C. Kalo, "ChronoSense: Exploring temporal understanding in large language models with time intervals of events," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025), Volume 2: Short Papers*. ACL, 2025, pp. 590–602. [Online]. Available: https://aclanthology.org/2025.acl-short.46/

[27] D. Xu, C. Ruan, E. Korpeoglu, S. Kumar, and K. Achan, "Inductive representation learning on temporal graphs," in *International Conference on Learning Representations (ICLR 2020)*, 2020. [Online]. Available: https://openreview.net/forum?id=rJeW1yHYwH

[28] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, "Temporal graph networks for deep learning on dynamic graphs," 2020, iCML

2020 Workshop on Graph Representation Learning and Beyond (GRL+). [Online]. Available: https://arxiv.org/abs/2006.10637

[29] J. Gastinger, S. Huang, M. Galkin, E. Loghmani, A. Parviz, F. Poursafaei, J. Danovitch, E. Rossi, I. Koutis, H. Stuckenschmidt, G. Rabusseau, and R. Rabbany, "TGB 2.0: A benchmark for learning on temporal knowledge graphs and heterogeneous graphs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.09639

[30] S. Xiong, A. Payani, R. Kompella, and F. Fekri, "Large language models can learn temporal reasoning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024), Volume 1: Long Papers*. ACL, 2024, pp. 10 452–10 470. [Online]. Available: https://aclanthology.org/2024.acl-long.563/

[31] J. Fang, B. Zhang, C. Wang, J. Wan, and Z. Xu, "Graph of verification: Structured verification of LLM reasoning with directed acyclic graphs," 2025. [Online]. Available: https://arxiv.org/abs/2506.12509

[32] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.04388

[33] Q. Lyu, S. Havaldar, A. Stein, L. Zhang, D. Rao, E. Wong, M. Apidianaki, and C. Callison-Burch, "Faithful chain-of-thought reasoning," in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL 2023), Volume 1: Long Papers*. ACL, 2023, pp. 305–329. [Online]. Available: https://arxiv.org/abs/2301.13379

[34] N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, P. West, C. Bhagavatula, R. Le Bras, J. D. Hwang, S. Sanyal, S. Welleck, X. Ren, A. Ettinger, Z. Harchaoui, and Y. Choi, "Faith and fate: Limits of transformers on compositionality," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.18654

[35] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models," 2024. [Online]. Available: https://arxiv.org/abs/2410.05229

[36] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: https://arxiv.org/abs/2305.20050

[37] J. Lee and J. Hockenmaier, "Evaluating step-by-step reasoning traces: A survey," 2025. [Online]. Available: https://arxiv.org/abs/2502.12289

[38] DeepSeek-AI, D. Guo, D. Yang, H. Zhang *et al.*, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[39] J. Chua and O. Evans, "Are DeepSeek R1 and other reasoning models more faithful?" in *ICLR 2025 Workshop on Foundation Models in the Wild*, 2025. [Online]. Available: https://arxiv.org/abs/2501.08156

[40] Ollama, "Ollama," https://github.com/ollama/ollama, 2024, software, accessed May 2026.

[41] Q. Ning, Z. Feng, and D. Roth, "A structured learning approach to temporal relation extraction," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. ACL, 2017, pp. 1027–1037. [Online]. Available: https://aclanthology.org/D17-1108/

[42] C. Yuan, Q. Xie, and S. Ananiadou, "Zero-shot temporal relation extraction with ChatGPT," in *Proceedings of the BioNLP 2023 Workshop at ACL 2023*. ACL, 2023, pp. 92–102. [Online]. Available: https://aclanthology.org/2023.bionlp-1.7/

[43] Q. Ning, H. Wu, and D. Roth, "A multi-axis annotation scheme for event temporal relations," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018), Volume 1: Long Papers*. ACL, 2018, pp. 1318–1328. [Online]. Available: https://aclanthology.org/P18-1122/

Appendix A

Ethics Self-Assessment

This project was assessed under the School of Computer Science student ethics self-assessment procedure. The project does not involve interacting with human subjects, collecting personal data, or any application with potential adverse consequences for human welfare. All datasets used are publicly available research corpora. No sensitive data is processed. The only ethical consideration identified was ensuring transparency in the automated verification system to avoid a black-box effect in reasoning assessment; this is addressed by the counterexample localisation and structured reporting described in Chapter 6.

The signed self-assessment form, approved by the project supervisor Dr Ognjen Arandjelović on 12 February 2026, is reproduced on the following page.

School of Computer Science
Student Project Ethics Form

Title of project

Verifying Language Model Outputs Using Temporal Graph Constraints: A Structured Evaluation Approach

Name of student

Hashim Iqbal (210002894)

Name of supervisor

Dr Ognjen Arandjelovic

To be completed by student

I confirm that:

(select one of the following options)

[1] ☒ My **intended project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my intended project.

[2] ☐ My **intended project** is covered by the Computer Science generic artifact evaluation ethical application (approval code CS15727).

[3] ☐ My **fall-back project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my fall-back project. I will submit a full ethics application for my intended project.

[4] ☐ My **fall-back project** raises no ethical issues: I have agreed with **all** of the statements on the following page, in relation to my fall-back project. My supervisor will submit an ethics amendment application for my intended project.

[5] ☐ My **fall-back project** is covered by approval CS15727. I will submit a full ethics application for my intended project.

[6] ☐ My **fall-back project** is covered by approval CS15727. My supervisor will submit an ethics amendment application for my intended project.

Signature of student

Date

12/02/2026

To be completed by supervisor

I confirm that I approve the student's statement above.

Signature of supervisor

Date

12/02/2026

Delete this page or leave blank if you are relying on the generic artifact evaluation approval

☒ I confirm that **all** of the following statements are correct, in relation to:

(select <u>one</u> of the following options)

☒ my intended project

☐ my fall-back project

- My project does **not** involve interacting with human subjects.
- My project does **not** involve collecting personal data on human subjects.
- My project does **not** have potential adverse consequences for human welfare and wellbeing.
- My project does **not** involve use of secondary data in a way that requires ethical review according to the University's guidance on secondary data.

> *Consider:*
>
> - *Will you survey, observe or interview human subjects?*
> - *Could your project have a significant negative effect on people in the study area?*
>
> *If your project involves secondary datasets, please list them with links in your DOER.*

- My project does **not** present any foreseeable risks to me or to any participants.

> *Consider:*
>
> - *Is there any potential for physical harm for anyone involved in the project?*
> - *Is there any potential for psychological harm, discomfort or stress for anyone involved in the project?*

- There are **no** potential conflicts of interest.

> *Consider:*
>
> - *Might research objectivity be compromised by sponsorship?*
> - *Might any issues of intellectual property or roles in research be raised?*

- My project is **not** funded externally, **or** the external funder appears on the Ethical Funder List on the UTREC website, **or** I have received ethical approval of the funder.

> *A project **cannot** commence with an unapproved external funder.*

- My project does **not** involve the use of living animals.

> *A fall-back project **cannot** involve the use of living animals. If your proposed full project involves the use of living animals, a proposal must be referred to the University's Animal Welfare and Ethics Committee (AWEC).*

Appendix B

Testing Summary

B.1 Overview

The test suite comprises 200 test functions distributed across 21 test modules. Running the suite from the project root with `PYTHONPATH=.` set produced 200 passes in the final validation run.

Tests are executed with:

```
cd stacs-temporal-graph-verification
PYTHONPATH=. python3 -m pytest tests/ -q
# Expected final validation result: 200 passed in 9.69s
```

The final validation run completed without warnings.

B.2 Test Modules and Coverage

Table B.1 lists each test module, its test count, and the component or behaviour it covers.

Table B.1: Test suite modules and coverage targets.

Module	n	**Coverage target**
`test_temporal_graph.py`	32	All public methods of `TemporalGraph`: acyclicity, transitive closure, simultaneity groups, contradiction detection, and the empty-graph edge case.
`test_evaluation.py`	25	Closure and direct-edge F1, symmetric canonicalisation, label normaliser lookup strategies (exact, case-insensitive, trigger-word), abstention flags, and over-commitment flags.
`test_prediction_schema.py`	21	Prediction parser, JSON repair pass, balanced-brace extractor, and all documented parse failure categories.
`test_axis_correlation.py`	13	Pearson and phi coefficient computation, constant-input handling, and binary agreement counts.

Continued on next page

Table B.1: Test suite modules and coverage targets (continued).

Module	n	**Coverage target**
`test_run_llm_baseline_naming.py`	12	Run directory naming, model-label sanitisation, timestamp format, and collision avoidance.
`test_correctness_correlation.py`	22	Spearman and point-biserial correlation, signal extraction from run records, and empty-gold task exclusion.
`test_constraints.py`	12	All eight invariant constraints plus the Category I LTL layer, verified against constructed `TemporalGraph` instances.
`test_genuine_ltl_formulas.py`	10	Category II LTL formulas: firing conditions, clear conditions, canonical temporal-edge support matching, and empty-step skip logic for both `ltl_unsupported_final_commitment` and `ltl_trace_inversion`.
`test_import_matres.py`	4	MATRES relation label mapping, TimeML document parsing, task construction from event pairs, and CLI invocation.
`test_import_maven_ere.py`	6	MAVEN-ERE relation mapping (including optional `OVERLAP`→`SIMULTANEOUS` coarsening), `valid` split conversion, and test-candidate generation.
`test_closure_coverage.py`	8	Coverage rate, committed-closure F1, and boundary conditions including all-UNKNOWN and empty-gold tasks.
`test_ltl.py`	4	LTL operators ($\mathbf{G}$, $\mathbf{F}$, $\mathbf{X}$, $\mathbf{U}$, $\wedge$, $\vee$, $\neg$), all five predicates, failure explanation traversal, and empty or single-state traces.
`test_trace.py`	2	Trace construction, cumulative `active_edges` semantics, predicate evaluation, and `with_violations` propagation.
`test_pairwise_eval.py`	3	Pairwise label derivation, confusion matrix construction, and verification-slice splits.
`test_dataset_and_validation.py`	6	JSONL schema validation, boolean coercion rejection, TempEval-3 record conversion, and canonical profile checks.
`test_run_summary.py`	3	Aggregate metric computation, bootstrap confidence interval structure, and report serialisation.
`test_structured_predictor.py`	2	Prompt construction, metadata output, and error propagation via mock `OllamaClient`.
`test_ollama_client.py`	3	Retry logic, exponential backoff, and transport error categorisation using mock HTTP responses.
`test_taxonomy.py`	5	Violation-to-category mapping for all documented violation types.
`test_run_llm_baseline_integration.py`	7	End-to-end gold-mode and noisy-mode pipeline (445 lines); `RunReport` fields, artefact structure, parse failure taxonomy, transport failure separation, and execution without a live Ollama server.
`test_run_model_sweep.py`	2	Sweep manifest loading, per-model run dispatch, and error continuation.
Total (active)	**200**	

B.3 Test Strategy and Debugging Approach

Test-driven development: New verification constraints, scoring functions, and analysis modules were accompanied by unit tests before being integrated into the main pipeline. This was particularly important for the verification logic, where failures are silent: an acyclicity check that misses a two-node cycle would distort the validity rate across the entire evaluation without producing a visible error.

Gold-mode integration testing: The `pred_source="gold"` mode in `BaselineRunConfig` bypasses LLM inference entirely and uses gold relations as predictions. This allows the full pipeline – parser, verifier, scorer, and report generation – to be exercised without a running Ollama server. The integration test (`test_run_llm_baseline_integration.py`, 445 lines, 5 test functions) uses this mode to assert fields of the resulting `RunReport`, including the exact formula violation type expected on contradiction-category tasks and the first-violation step histogram.

Pipeline isolation modes: The `pred_source` flag supports four modes for debugging specific pipeline stages:

- `gold`: use gold relations as predictions; isolates the verifier and scorer from inference failures.
- `empty`: predict nothing on every task; establishes the lower bound and verifies empty-prediction handling.
- `noisy`: randomly corrupt gold relations at a configurable rate; useful for testing metric sensitivity.
- `llm`: live Ollama inference (default).

Scope of testing: Live LLM inference correctness is not tested. Testing that a specific model produces a specific output on a specific prompt is not meaningful as a regression test because the framework's purpose is to measure model behaviour, not to assert it. `test_structured_predictor.py` instead tests prompt construction, metadata, and error propagation using a mock `OllamaClient` that returns fixed strings.

Regression safety: Each time a module was modified in response to empirical findings or supervisor feedback, the full suite was run to confirm that no existing behaviour had changed in an unintended way. This was essential when coverage-aware closure F1 was added (affecting the aggregate reporting path), when the Category II LTL formulas were introduced (affecting `is_valid` via formula violations), and when the MATRES and MAVEN-ERE import adapters were added.

Appendix C

User Manual

C.1 Repository and Attribution

The artefact is hosted at:

`https://github.com/Haz-ctrl/stacs-temporal-graph-verification`

The framework depends on the Ollama inference server [40] for live LLM runs. The evaluation datasets are described in Section C.5 with full provenance.

C.2 Prerequisites

- **Python 3.12** or later.
- **Ollama** (`https://ollama.com`), installed and running locally. The default server address is `http://localhost:11434`.
- **Git** for cloning the repository.

C.3 Installation

```
git clone https://github.com/Haz-ctrl/stacs-temporal-graph-verification.git
cd stacs-temporal-graph-verification

pip install -r requirements.txt

# Verify the installation
PYTHONPATH=. python3 -m pytest tests/ -q
# Expected: 200 passed
```

All scripts must be run from the repository root with `PYTHONPATH=.` set so that the `src` and `scripts` packages are importable.

C.4 Lab Machine Setup

On University of St Andrews School of Computer Science lab machines, Ollama must be installed to the user home directory rather than system-wide. The following sequence installs Ollama, starts the server, and verifies the environment before any inference run.

Step 1 — Install Ollama:

```
bash scripts/lab_install_ollama.sh # installs Ollama 0.17.6 by default
# To install a specific version:
bash scripts/lab_install_ollama.sh 0.18.0
```

The script downloads the AMD64 and ROCm tarballs from the Ollama GitHub releases page, extracts them to `~/opt/ollama-vX.Y.Z/`, and points the symlink `~/opt/ollama` at the new installation. Cached tarballs are stored under `~/src/ollama/` and are reused if present.

Step 2 — Start the Ollama server:

```
nohup ollama serve > ~/ollama_serve.log 2>&1 &
```

The server runs in the background and writes logs to `~/ollama_serve.log`. Allow a few seconds for it to become reachable before proceeding.

Step 3 – Activate the environment:

```
source scripts/ollama_env.sh
```

This script adds `~/opt/ollama/bin` to `PATH`, verifies the Ollama binary is found, checks that the server is reachable at `http://localhost:11434`, and prints the list of currently available models. If the server is not reachable, the script prints the `nohup ollama serve` command as a reminder.

Step 4 – Pull models and run: After the environment check passes, pull any required models and proceed with the inference and sweep commands in Sections C.6–C.7.

```
ollama pull deepseek-r1:7b
ollama pull qwen3.5:9b
ollama pull llama3.1:8b
ollama pull mistral:7b
ollama pull gemma3:12b
```

The lab environment is configured for GPU-accelerated inference via ROCm; the installed tarball includes the ROCm backend automatically. If a model run stalls, increase `-timeout-s` to 600 and check `~/ollama_serve.log` for OOM or device errors.

C.5 Dataset Provenance and Preparation

C.5.1 TempEval-3

The TempEval-3 Platinum corpus is the primary external benchmark used in this project. It was introduced by UzZaman et al. as SemEval-2013 Task 1 and provides event–event temporal relation annotations over English newswire [3]. The temporal awareness F1 metric used for closure-level evaluation is defined in the companion paper by UzZaman and Allen [19].

A pre-processed evaluation set derived from the Platinum split is committed to the repository at `data/tempeval_eval.jsonl` (496 tasks) and can be used directly without any additional download. The original host URL for the raw TempEval-3 corpus is no longer available; the SemEval-2013 Task 1 shared task is documented in the ACL Anthology at `https://aclanthology.org/S13-2001/`. Researchers requiring the raw TimeML files for reconstruction or extended experiments should contact the task organisers via the ACL Anthology proceedings entry or obtain the underlying TimeBank corpus from the Linguistic Data Consortium (LDC2006T08).

To rebuild `tempeval_eval.jsonl` from raw TimeML if the corpus is obtained independently:

```
PYTHONPATH=. python3 scripts/import_tempeval3.py \
    --input-root /path/to/TempEval3-corpus \
    --split test \
    --output data/tempeval_eval.jsonl \
    --stats-out data/tempeval_eval_stats.json \
    --category tempeval_relation
```

The `-split` argument may be repeated to include multiple splits. The adapter coarsens the six-label TimeML scheme to the framework's four-label schema: IBEFORE → BEFORE, IAFTER → AFTER, and IDENTITY → SIMULTANEOUS; INCLUDES and DURING are skipped and counted in the adapter statistics.

C.5.2 MAVEN-ERE

MAVEN-ERE is a large-scale dataset for event temporal (and causal, coreference, and subevent) relation extraction [9]. The raw JSONL files are available from two mirrors:

- Google Drive (provided by the dataset authors):
  `https://drive.google.com/file/d/1fxomYO6zPl5DDrDr_HeWFK14s8BpW1z-/view`
- Tsinghua Cloud:
  `https://cloud.tsinghua.edu.cn/f/a7d1db6c44ea458bb6f0/`

Place the downloaded `MAVEN_ERE.zip` contents under `data/raw/MAVEN_ERE/` so that the directory contains `train.jsonl`, `valid.jsonl`, and `test.jsonl`.

The framework imports only EVENT–EVENT pairs and retains BEFORE and SIMULTANEOUS labels; the fuller MAVEN-ERE temporal inventory (OVERLAP, CONTAINS, BEGINS-ON, ENDS-ON) is outside the current four-label schema. To convert the `valid` split into a stratified evaluation slice:

```
PYTHONPATH=. python3 scripts/import_maven_ere.py \
    --input data/raw/MAVEN_ERE/valid.jsonl \
    --split valid \
    --output data/maven_ere_balanced_2to1.jsonl \
    --stats-out data/maven_ere_balanced_2to1_stats.json \
    --category maven_ere_temporal \
    --context-radius 1 \
    --before-multiplier 2.0 \
    --seed 42
```

Key optional flags:

```
--coarsen-overlap # Map MAVEN-ERE OVERLAP to SIMULTANEOUS
--no-event-event-only # Also include EVENT-TIMEX pairs
--max-tasks 500 # Hard cap after stratified sampling
```

To generate unlabelled test candidates for submission to the MAVEN-ERE shared task leaderboard:

```
PYTHONPATH=. python3 scripts/import_maven_ere.py \
    --input data/raw/MAVEN_ERE/test.jsonl \
    --split test \
    --output data/maven_ere_test_candidates.jsonl \
    --context-radius 1 \
    --test-sentence-window 1
```

C.5.3 MATRES (future work)

MATRES (Multi-Axis Temporal RElations for Start-points) is a high-quality event temporal annotation built over TimeBank and AQUAINT, introduced by Ning, Wu, and Roth [43]. It uses four labels: BEFORE, AFTER, EQUAL, and VAGUE, which map directly to the framework's four-label schema (EQUAL → SIMULTANEOUS, VAGUE → UNKNOWN).

A balanced evaluation set of 400 tasks (100 per canonical label) is committed to the repository at `data/matres_balanced_small.jsonl` and can be used immediately without any additional download. The relation annotations were sourced from the MATRES GitHub repository maintained by Ning [43] at:

`https://github.com/qiangning/MATRES`

To run the five-model sweep on the committed MATRES set:

```
PYTHONPATH=. python3 scripts/run_model_sweep.py \
    --manifest manifests/lab_models.json \
    --data data/matres_balanced_small.jsonl \
    --output-root outputs/runs/matres_full \
    --analysis-out outputs/analysis/matres_full \
    --timeout-s 120 \
    --max-retries 4 \
    --retry-backoff-s 3.0 \
    --continue-on-error
```

To rebuild `matres_balanced_small.jsonl` from source, the import adapter requires the raw MATRES relation files and the corresponding TimeML documents. The relation files are available directly from the MATRES GitHub repository; the TimeML source documents originate from the TimeBank and AQUAINT corpora distributed by the Linguistic Data Consortium.

```
PYTHONPATH=. python3 scripts/import_matres.py \
    --matres-input path/to/MATRES/timebank.txt \
    --matres-input path/to/MATRES/aquaint.txt \
    --timeml-root path/to/timeml_documents/ \
    --output data/matres_balanced_small.jsonl \
    --stats-out data/matres_balanced_small_stats.json \
    --category matres_temporal \
    --max-per-label 100 \
    --seed 42
```

The adapter accepts multiple `-matres-input` files, handles MATRES `VAGUE` → `UNKNOWN` mapping, and coerces `EQUAL` → SIMULTANEOUS.

C.5.4 Canonical Synthetic Set

The controlled synthetic dataset is author-constructed and included in the repository at `data/temporal_reasoning_eval.jsonl`. It comprises 250 tasks in five categories: 60 `linear_chain`, 50 `transitive_reasoning`, 30 `long_chain`, 60 `ambiguous`, and 50 `contradiction`. To regenerate it from scratch with the same configuration:

```
PYTHONPATH=. python3 scripts/generate_temporal_dataset.py \
    --out data/temporal_reasoning_eval.jsonl \
    --n-linear 60 \
    --n-transitive 50 \
    --n-long 30 \
    --n-ambiguous 60 \
    --n-contradiction 50 \
    --seed 42
```

C.6 Quick Start: Single Model Run

The following command runs a single model over the TempEval-3 evaluation set:

```
PYTHONPATH=. python3 scripts/run_llm_baseline.py \
    --data data/tempeval_eval.jsonl \
    --model gemma3:12b \
    --output-root outputs/runs/tempeval_full
```

Key optional arguments:

```
--max-tasks 20 # Limit to first 20 tasks (quick test)
--temperature 0.0 # Decoding temperature (default: 0.0)
--seed 42 # Random seed (default: 42)
--timeout-s 300 # Per-request read timeout in seconds
--max-retries 4 # Transport retry attempts (default: 4)
--retry-backoff-s 3.0 # Exponential backoff base in seconds
--pred-source gold # Use gold labels -- no LLM needed
--log-raw # Save raw model output in predictions.jsonl
--validate-data # Validate dataset schema before running
--strict-data # Tighten dataset validation
```

Before spending time on LLM runs, use gold mode to confirm the pipeline is working correctly:

```
PYTHONPATH=. python3 scripts/run_llm_baseline.py \
    --data data/tempeval_eval.jsonl \
    --pred-source gold \
    --output-root outputs/runs/sanity_check
# Should produce near-perfect scores
```

C.7 Running a Multi-Model Sweep

To run all five models sequentially against the same dataset using the pre-configured sweep manifest at `manifests/lab_models.json`:

```
PYTHONPATH=. python3 scripts/run_model_sweep.py \
    --manifest manifests/lab_models.json \
    --data data/tempeval_eval.jsonl \
    --output-root outputs/runs/tempeval_full \
    --analysis-out outputs/analysis/tempeval_full \
    --timeout-s 120 \
    --max-retries 4 \
    --retry-backoff-s 3.0 \
    --continue-on-error
```

For MAVEN-ERE:

```
PYTHONPATH=. python3 scripts/run_model_sweep.py \
```

```
    --manifest manifests/lab_models.json \
    --data data/maven_ere_balanced_2to1.jsonl \
    --output-root outputs/runs/maven_ere_full \
    --analysis-out outputs/analysis/maven_ere_full \
    --timeout-s 300 \
    --max-retries 4 \
    --retry-backoff-s 3.0 \
    --continue-on-error
```

The `-continue-on-error` flag records failures in `sweep_index.json` and continues with remaining models rather than aborting. The `-skip-analysis` flag suppresses post-sweep summarisation if analysis should be run separately.

The manifest is a JSON list of model entries; the checked-in `lab_models.json` contains all five models evaluated in the dissertation with per-model timeout and retry overrides.

C.8 Generating Analysis and Plots

To aggregate results from a completed sweep into summary tables and plots:

```
PYTHONPATH=. python3 scripts/summarise_runs.py \
    --runs outputs/runs/tempeval_full \
    --out outputs/analysis/tempeval_full
```

The `-runs` argument accepts one or more directories and scans for timestamped run subdirectories automatically. Output files:

```
outputs/analysis/{dataset}/
    summary.csv # Per-run aggregate metrics
    category_breakdown.csv # Per-category fidelity and consistency
    difficulty_breakdown.csv # Structural complexity slices
    failure_breakdown.csv # Parse and verification failure rates
    counterexamples.md # Qualitative failure examples
    report.md # Narrative markdown summary
    axis_correlation_*.png # Per-model intrinsic axis heatmaps
    plots/ # Bar charts and heatmaps
```

To generate the supplementary dissertation plots (cross-dataset comparison, violation heatmaps, RQ3 correlations, and 7 others):

```
PYTHONPATH=. python3 scripts/generate_analysis_plots.py \
    --canonical-dir outputs/runs/canonical_full \
    --tempeval-dir outputs/runs/tempeval_full \
    --maven-ere-dir outputs/runs/maven_ere_full \
    --canonical-analysis outputs/analysis/canonical_full \
    --tempeval-analysis outputs/analysis/tempeval_full \
    --maven-ere-analysis outputs/analysis/maven_ere_full \
    --out outputs/analysis/supplementary_plots
```

The script produces 10 figures and a `rq3_spearman_correlations.csv`; all are written to `-out`.

C.9 Rescoring Existing Predictions

After updating the verifier, existing prediction files can be rescored without re-running LLM inference:

```
python3 scripts/rescore_run.py \
    --predictions outputs/runs/tempeval_full/<run_id>/predictions.jsonl
# Writes: predictions_rescored.jsonl beside the original
```

To specify a different output path:

```
python3 scripts/rescore_run.py \
    --predictions outputs/runs/tempeval_full/<run_id>/predictions.jsonl \
    --output outputs/runs/tempeval_full/<run_id>/predictions_v2.jsonl
```

The script prints: records rescored, parse failures skipped, and the count of tasks where `is_valid` changed relative to the original run.

C.10 Validating a Dataset

Before running inference on a new dataset:

```
PYTHONPATH=. python3 scripts/validate_dataset.py \
    --data data/my_dataset.jsonl \
    --profile generic
```

Use `-profile canonical` for the synthetic evaluation format, which additionally checks that category labels are drawn from the expected set and that `expected_valid` and `expected_consistent` fields are present. Use `-require-expected-fields` to enforce this check on any profile. Use `-strict` to treat warnings as errors. Use `-out report.json` to write a machine-readable validation report.

C.11 Browser-Based Run Inspector

A self-contained HTML run explorer is provided at `tools/verifier_explorer.html` and requires no server:

```
open tools/verifier_explorer.html # macOS
# On Linux: xdg-open tools/verifier_explorer.html
```

Click **Load predictions.jsonl** and select any `predictions.jsonl` from a run directory. The explorer shows: a task list with category chip, pass/fail dot, and free-text filter; prediction detail with events and colour-coded gold/predicted relations; a verification panel with violations by type and the first-violation step highlighted in the trace; and aggregate statistics including validity rate, mean F1, and top violation types.

C.12 Dataset JSONL Format

Each line of a dataset file is a JSON object with the following fields:

```
{
  "id": "unique_task_identifier",
  "category": "tempeval_relation",
  "question": "Passage text and event mentions...",
  "events": ["event text [ei1]", "event text [ei2]"],
  "gold_relations": [
    ["event text [ei1]", "event text [ei2]", "BEFORE"]
  ],
  "expected_valid": true,
  "expected_consistent": true
}
```

- `id`: a unique string identifying the task.
- `category`: a label used for per-category reporting (e.g., `linear_chain`, `tempeval_relation`, `contradiction`, `maven_ere_temporal`, `matres_temporal`).
- `question`: the natural language prompt presented to the model.
- `events`: the allowed event list; models are instructed to copy these strings exactly.
- `gold_relations`: a list of `[source, target, label]` triples. May be empty for ambiguous or underspecified tasks.
- `expected_valid`: `true` if the gold annotation is intrinsically consistent (used for validity-expectation alignment scoring).
- `expected_consistent`: `true` if the task is expected to have a determinate answer; set to `false` for contradiction tasks.

C.13 Output Structure

A completed run produces the following artefacts under the timestamped run directory:

```
outputs/runs/{dataset}/{model}_{timestamp}/
    predictions.jsonl # Per-task predictions with verification
    predictions_rescored.jsonl # Rescored predictions (if rescore run)
```

```
    report.json # Full aggregate run report
    run_manifest.json # Configuration and provenance record
```

The `run_manifest.json` records: dataset path and SHA-256 hash, model tag, seed, temperature, prediction source, and the git commit SHA of the code at run time.

C.14 Troubleshooting

Transport timeouts: If inference stalls, increase the timeout and retry count: `-timeout-s 600 -max-retries 4 -retry-backoff-s 5.0`. The Ollama server must remain running throughout the sweep; long inter-request gaps can cause the server to drop the connection.

Parse failures: If a model produces consistently malformed output (reported as `invalid_json` in the run summary), inspect raw output with `-log-raw` to diagnose the failure pattern. Markdown fence wrapping and preamble text are the most common patterns; both are handled by the extractor and repair pass, but some models require prompt adjustments.

Module not found: All scripts must be run with `PYTHONPATH=.` from the repository root. Running from any other directory or without the flag produces `ModuleNotFoundError: No module named 'src'`.

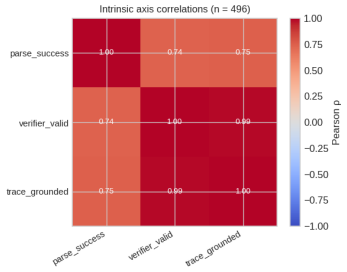
Ollama model not available: If a model sweep fails on a specific model, confirm availability with `ollama list` and pull the model if absent. The `-continue-on-error` flag allows the sweep to skip a failing model and continue with the remaining entries.

Appendix D

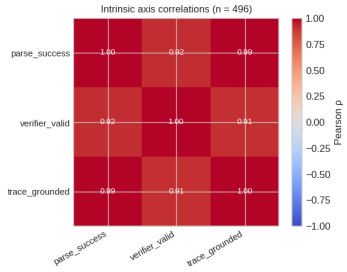
Intrinsic Axis Correlation Matrices

This appendix provides the full pairwise Pearson correlation matrices across the three intrinsic binary axes (`parse_success`, `verifier_valid`, `trace_grounded`) for all five models on each of the three evaluation datasets. High correlation ($|\rho| > 0.90$) indicates that two axes provide largely redundant signal on that dataset; low correlation indicates genuine separation. The matrices complement the collinearity discussion in Section 7.6.2.

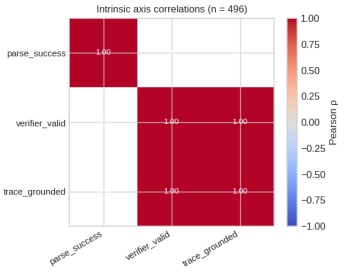
TempEval-3



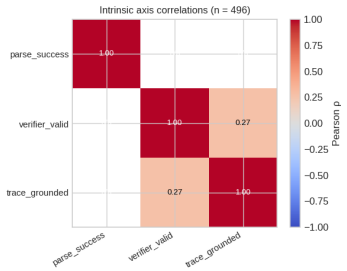
(a) DeepSeek R1 7B
`valid`–`grounded`: $\rho = 0.99$



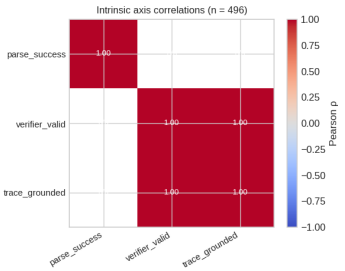
(b) Qwen 3.5 9B
`valid`–`grounded`: $\rho = 0.91$



(c) Llama 3.1 8B
`valid`–`grounded`: $\rho = 1.00$



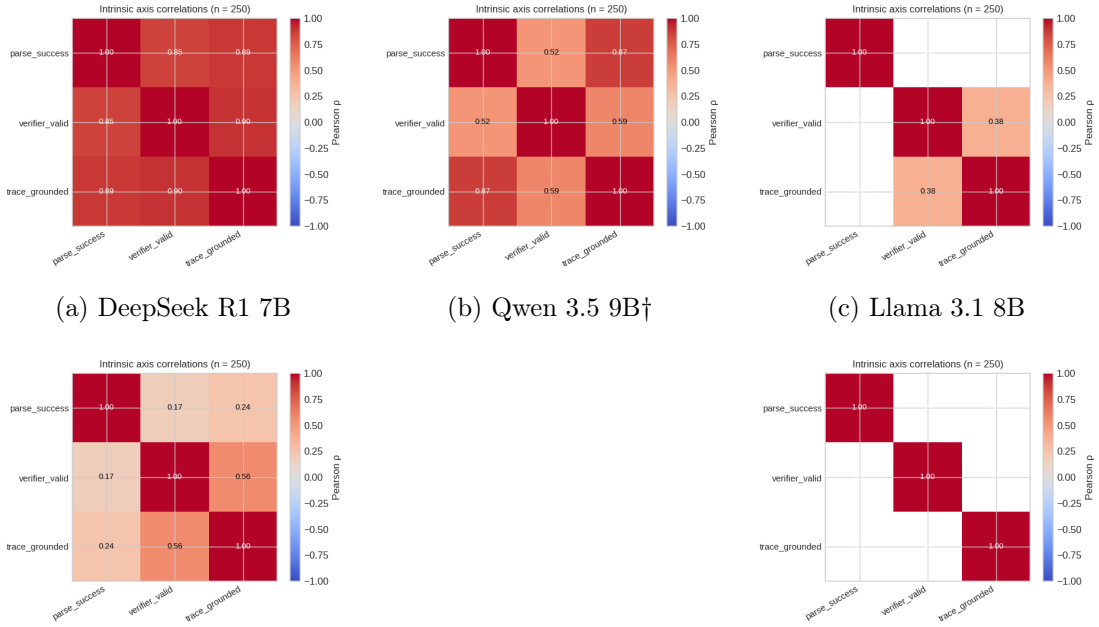
(d) Mistral 7B
`valid`–`grounded`: $\rho = 0.27$



(e) Gemma 3 12B
`valid`–`grounded`: $\rho = 1.00$

Figure D.1: Intrinsic axis correlations on TempEval-3 ($n = 496$ per model). Four of five models show `verifier_valid`–`trace_grounded` correlation above 0.90. Mistral 7B is the exception ($\rho = 0.27$), reflecting its high genuine LTL violation rate on this dataset: many predictions are trace-grounded but fail the LTL layer.

Canonical Synthetic Set

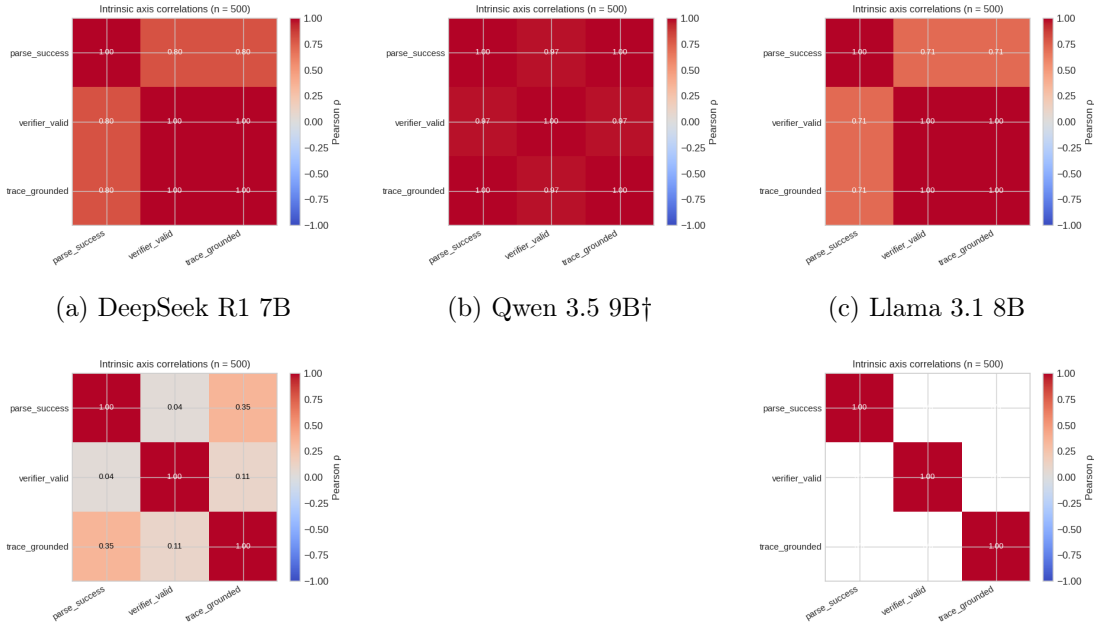


(a) DeepSeek R1 7B (b) Qwen 3.5 9B† (c) Llama 3.1 8B

(d) Mistral 7B (e) Gemma 3 12B

Figure D.2: Intrinsic axis correlations on the canonical synthetic set ($n = 250$ per model). The five-category structure (including contradiction and ambiguity items) produces more variation in all three axes than the pairwise TempEval-3 tasks. Llama 3.1 8B shows the lowest `valid`–`grounded` correlation ($\rho = 0.38$) on this dataset, reflecting its high contradiction-detection rate: it correctly produces invalid structures on contradiction items whilst remaining trace-grounded. The † symbol indicates Qwen's transport failure rate of 0.084 on this dataset.

MAVEN-ERE



(a) DeepSeek R1 7B

(b) Qwen 3.5 9B†

(c) Llama 3.1 8B

(d) Mistral 7B
`parse`–`valid`: $\rho = 0.04$

(e) Gemma 3 12B
all axes $\rho = 1.00$

Figure D.3: Intrinsic axis correlations on MAVEN-ERE ($n = 500$ per model). The Mistral 7B plot (d) is the only case across all fifteen dataset–model combinations where `parse_success` and `verifier_valid` are genuinely dissociated ($\rho = 0.04$): Mistral parses 99.8% of tasks but is structurally invalid on 57.7% of them. Gemma 3 12B (e) shows perfect correlation across all axes because its validity is constant at 1.000 on this dataset. The † symbol indicates Qwen's transport failure rate of 0.130 on this dataset.